

ПОЛНОЕ ПОШАГОВОЕ РЕШЕНИЕ

заданий 1–7 демонстрационного экзамена

Специальность 09.02.07
КИМ 09.02.07-5-2027

Выполнил(а): _____
Дата: 07 октября 2026 г.

Как пользоваться документом

В каждой главе после результата есть раздел «Как выполнить на практике»: какие файлы открыть, какую команду запустить, что именно ввести/проверить и какой ответ ожидать. Приведены копируемые команды, SQL, API-запросы и формулы Excel. В приложениях даны полные листинги исходников; готовые ER-PDF, SQLite, XLSX и DOCX лежат в каталогах solutions/task1_er ... task7_analysis.

Показатель	Результат
Себестоимость заказа №1	11448,12 руб.
Строк задания 7	30; удалено 0; дублей 0
Годовая / средняя месячная выручка	7759680 / 646640 руб.
Максимум / минимум	Ноябрь — 1794540 / Сентябрь — 127949 руб.
Автотесты	7 пройдены

Задание 1. Проектирование ER-диаграммы

Шаг 1. Изучить формы заказа покупателя, заказа на производство, спецификацию, цены, расчёт стоимости и JSON-справочники. Выделить сущности заказчика, заказа, строки заказа, товара, ресурса, спецификации, цены, скидки, производственного заказа и его строк.

Шаг 2. Задать первичные ключи. Для документа — отдельный идентификатор; строки заказа имеют собственный PK и FK на заказ/товар. Клиентский ID хранится как текст, чтобы не терять ведущие нули.

Шаг 3. Установить связи: customer 1:N customer_orders; customer_orders 1:N order_items; products 1:N order_items; products M:N resources через product_specification; production_orders 1:N production_order_items.

Шаг 4. В resources объединить материалы и операции, различая их значением resource_type. В спецификации хранить количество ресурса на одну единицу изделия.

Шаг 5. Для цен/скидок предусмотреть два nullable FK, но CHECK разрешает заполнить только один: ссылка либо на product, либо на resource. Цена/скидка в строке заказа при необходимости является историческим снимком и не должна зависеть от текущего каталога.

Шаг 6. Обеспечить ЗНФ: атрибуты заказчика не повторяются в заказе; имя ресурса не дублируется в BOM; себестоимость рассчитывается запросом и не хранится как дублируемое поле.

Шаг 7. Представить модель как ER_диаграмма.pdf; редактируемый исходник — ER-модель.mmd. Полный DDL отражён в schema.sql.

Сущности модели

Таблица	Ключи и смысл
customers	PK customer_id; контрагенты
customer_orders	PK order_id; FK customer_id
order_items	PK строки; FK заказ/товар; qty, snapshot price/discount
products	PK product_id; каталог и заглушки
resources	PK resource_id; материал/операция
product_specification	PK; FK товар/ресурс; норма
prices / discounts	ровно одна FK-ссылка на товар или ресурс
production_orders / items	производственный заказ и строки выпуска

Кардинальности: customer 1:N orders; order 1:N order_items; product 1:N order_items; product M:N resource через specification; production order 1:N production items.

В источниках повторяются product 887, resource 456, order-item IDs 31/81, discount ID 15. Товары 398/881/942 отсутствуют в справочнике. Дубли сохранены surrogate keys, отсутствующие товары помечены заглушками; FK не отключались.

Диаграмма: task1_er/ER_диаграмма.pdf; редактируемая модель: task1_er/ER-модель.mmd.

Как выполнить на практике

Действие 1. Откройте Заказ покупателя.xlsx и Заказ на производство.xlsx; для справочников откройте Заказчики.json, Товары.json, Материалы.json, Спецификация.xlsx и Цены.xlsx. Выпишите поля шапки отдельно от повторяющихся строк документа: у покупательского заказа есть один заказчик и несколько строк товара; у спецификации каждому изделию соответствует несколько ресурсов.

Действие 2. Нарисуйте десять таблиц, перечисленных в таблице «Сущности модели». В каждой таблице подчеркните PK, у каждой связи отметьте FK. Не переносите ФИО/адрес заказчика в каждую строку заказа: храните эти поля один раз в customers, а в customer_orders оставьте только customer_id.

Действие 3. Соедините таблицы так: customers 1:N customer_orders; customer_orders 1:N order_items; products 1:N order_items. Между products и resources получится M:N, поэтому вставьте связующую product_specification и положите в неё quantity_per_product — норму на одну единицу продукции.

Действие 4. Материалы и операции поместите в resources, а различайте их значением resource_type. В prices и discounts нарисуйте два возможных nullable FK (product_id, resource_id) и подпишите правило: заполнен ровно один. В производственном документе отделите шапку production_orders от строк production_order_items.

Действие 5. Проверьте ЗНФ на каждом поле: значение должно описывать ключ, весь ключ и только ключ. Если название ресурса повторяется в строках BOM — удалите повтор и оставьте resource_id; если заказчик повторяется в заказах — вынесите его в customers. Затем сохраните редактируемую модель в ER-модель.mmd.

Действие 6. Чтобы получить сдаваемый PDF, запустите генератор из корня репозитория. Если меняете состав таблиц, обновите и список сущностей в generate_diagram.py, потому что генератор рисует PDF по этому списку, а не компилирует Mermaid автоматически.

Готовые объявления связей для блока erDiagram в ER-модели:

```
erDiagram
    CUSTOMERS ||--o{ CUSTOMER_ORDERS : оформляет
    CUSTOMER_ORDERS ||--o{ ORDER_ITEMS : содержит
    PRODUCTS ||--o{ ORDER_ITEMS : заказан
    PRODUCTS ||--o{ PRODUCT_SPECIFICATION : состоит_из
    RESOURCES ||--o{ PRODUCT_SPECIFICATION : входит_в
    PRODUCTS o|--o{ PRICES : цена_товара
    RESOURCES o|--o{ PRICES : цена_ресурса
    PRODUCTS o|--o{ DISCOUNTS : скидка_товара
    RESOURCES o|--o{ DISCOUNTS : скидка_ресурса
    PRODUCTION_ORDERS ||--o{ PRODUCTION_ORDER_ITEMS : содержит
    PRODUCTS ||--o{ PRODUCTION_ORDER_ITEMS : выпускается
```

Команда построения ER-диаграмма.pdf:

python3 solutions/task1_er/generate_diagram.py

Результат: solutions/task1_er/ER-диаграмма.pdf и редактируемый solutions/task1_er/ER-модель.mmd. На диаграмме должны быть видны имена таблиц, атрибуты, PK/FK и все кардинальности.

Задание 2. Создание и заполнение базы данных

Шаг 1. Создать SQLite-БД по schema.sql. Включить PRAGMA foreign_keys=ON, определить NOT NULL, CHECK, FK и индексы по FK.

Шаг 2. Запустить seed.py. Он читает Заказчики.json, Заказы.json, Позиции_заказов.json, Товары.json, Материалы.json, Спецификация.json, Цены.json и Скидки.json и загружает их одной транзакцией. Обязательный импорт Заказчики.json выполняется.

Шаг 3. Сопоставить JSON-поля с колонками, например address из источника записать как address. Числовые ID преобразовать в INTEGER, а customer_id оставить TEXT.

Шаг 4. Обработать конфликтные ссылки без отключения FK: повторные source ID сохранить отдельными surrogate PK с полем source_*_id. Отсутствующие товары 398, 881, 942 представить явно помеченными заглушками.

Шаг 5. Проверить PRAGMA foreign_key_check; ожидаемый результат — пустой набор нарушений. Проверить количество строк в таблицах и сохранить manufacturing.sqlite3.

Команда: python3 solutions/task2_database/seed.py. Скрипт перезагружает данные идиоматично при той же схеме.

Таблица	Строк
customers	6
products	132
resources	103
product_specification	74
customer_orders	70
order_items	70
prices	60
discounts	51

После загрузки PRAGMA foreign_key_check не находит нарушений. Снимок цены клиентской позиции оставлен NULL, поскольку JSON позиции не передаёт цену; скидка по умолчанию 0.

Как выполнить на практике

Действие 1. Положите исходные JSON в корень репозитория (в данном наборе они уже там). Откройте schema.sql: сначала выполняются CREATE TABLE, затем индексы. Внешние ключи включаются через PRAGMA foreign_keys=ON; NOT NULL запрещает пустые обязательные поля; CHECK ограничивает количество положительным числом, тип ресурса — заданными вариантами.

Действие 2. Из корня проекта запустите seed.py с явными путями. Скрипт проверит наличие восьми JSON, прочтёт их как UTF-8, выполнит schema.sql, очистит дочерние таблицы перед повторной загрузкой и вставит данные транзакционно.

Действие 3. Посмотрите на функцию, сопоставляющую поля: customer_id приводится к str, потому что это текстовый идентификатор; адрес читается сначала из исходного ключа address, а при его отсутствии — из address. Для товара и ресурса создаются карты source ID -> внутренний PK, чтобы ссылки можно было восстановить.

Действие 4. До вставки проверьте конфликтные ключи в напечатанных предупреждениях. В этих JSON товары 398/881/942 отсутствуют в справочнике, source product_id=887 и resource_id=456 повторяются, а ID строк заказа 31/81 и скидки 15 не уникальны. Не выключайте FK и не удаляйте записи: импортёр добавляет помеченные заглушки или surrogate PK и сохраняет исходный номер в source_*_id.

Действие 5. После импорта выполните PRAGMA foreign_key_check и сравните количество строк с контрольной таблицей ниже. Если есть нарушение FK, не переходите к заданию 3: сначала исправьте карту ID/ссылку и повторите импорт.

Создать/перезагрузить базу в указанном месте:

```
python3 solutions/task2_database/seed.py --data-dir . --db-path solutions/task2_database/manufacturing.sqlite3
```

Проверить внешние ключи и число записей из Python:

```
import sqlite3

db = sqlite3.connect('solutions/task2_database/manufacturing.sqlite3')
print('foreign_key_check =', db.execute('PRAGMA foreign_key_check').fetchall())
for table in ('customers', 'products', 'resources', 'product_specification',
              'customer_orders', 'order_items', 'prices', 'discounts'):
    print(table, db.execute(f'SELECT COUNT(*) FROM {table}').fetchone()[0])
db.close()
```

Успешная проверка выводит пустой список foreign_key_check и количества: customers 6, products 132, resources 103, product_specification 74, customer_orders 70, order_items 70, prices 60, discounts 51. В order_items цена из источника не передавалась, поэтому unit_price_at_order = NULL, а discount_per_unit = 0.

Задание 3. Запрос себестоимости заказа

Шаг 1. Передать номер заказа как параметр :order_id. Получить заказанные товары и количество из customer_orders и order_items.

Шаг 2. По product_id найти спецификацию. Каждая строка задаёт норму ресурса на одну единицу товара.

Шаг 3. По resource_id подобрать цену ресурса, включая и материалы, и технологические операции. Цена готового продукта и скидка покупателю не являются производственными затратами.

Шаг 4. Для каждого ресурса вычислить quantity заказа × quantity_per_product × unit_price; просуммировать, округлить до 2 знаков.

Шаг 5. Сравнить количество позиций спецификации и позиций с найденной ценой. Если цена или BOM отсутствуют, запрос возвращает NULL, а не заниженный частичный итог.

Полный итоговый запрос и запрос детализации приложены в task3_query/order_cost.sql.

Проверочный расчёт заказа №1

Ресурс	Норма	Цена	На заказ
Евровинт 6,5x5	0,012	432,083333	10,37
Мебельная деталь 500x800	2	37,50	150,00
Мебельная деталь 600x800	4	17,125	137,00
Опора	4	145,71875	1165,75
Столешница круглая	1	1242,50	2485,00
Распил ДСП, МДФ и листового материала	1	450,00	900,00
Сборка модулей	1	1400,00	2800,00
Упаковка	1	1900,00	3800,00
ИТОГО	—	—	11448,12

$10,37 + 150,00 + 137,00 + 1165,75 + 2485,00 + 900,00 + 2800,00 + 3800,00 = 11448,12$ руб.

Как выполнить на практике

Действие 1. Сначала выберите номер заказа в customer_orders. Для каждой его строки возьмите oi.quantity; по oi.product_id найдите все строки product_specification; для каждой строки спецификации получите resource_id, норму quantity_per_product и цену unit_price ресурса. Тип ресурса не фильтруйте: в сумму входят и материалы, и операции.

Действие 2. Для одной позиции заказа вычислите стоимость каждого компонента по формуле количество заказа × норма ресурса на изделие × цена ресурса. Сложите все компоненты всех товарных строк, округлите итог один раз до двух знаков. Не прибавляйте цену продажи готового товара и не вычитайте покупательскую скидку — это производственная себестоимость.

Действие 3. Выполните первый запрос из order_cost.sql как параметризованный SQL. Значение передавайте словарём, а не склеивайте в текст SQL: это исключает ошибки кавычек и SQL-инъекции. Если запрос вернул NULL или priced_positions меньше required_cost_positions, проверьте BOM/цену; не сдавайте частичную сумму.

Действие 4. Чтобы объяснить результат, выполните второй SELECT из того же файла: он выводит по каждой паре заказ-ресурс норму, цену и component_total. Для заказа №1 сверка идёт по восьми строкам расчёта из таблицы ниже.

Запустить итоговый параметризованный запрос для заказа №1:

```
from pathlib import Path
import sqlite3

path = Path('solutions/task3_query/order_cost.sql')
text = path.read_text(encoding='utf-8')
total_sql = text.split('-- Детализация того же расчёта: ', 1)[0]
db = sqlite3.connect('solutions/task2_database/manufacturing.sqlite3')
print(db.execute(total_sql, {'order_id': 1}).fetchone())
# Ожидается: (1, 11448.12, 8, 8)
```

Вывести построчную расшифровку по каждому ресурсу (самостоятельный запуск):

```

from pathlib import Path
import sqlite3
text = Path('solutions/task3 query/order_cost.sql').read_text(encoding='utf-8')
detail_sql = text.split('-- Детализация того же расчёта:', 1)[1]
detail_sql = detail_sql[detail_sql.index('SELECT'):]
with sqlite3.connect('solutions/task2_database/manufacturing.sqlite3') as db:
    for row in db.execute(detail_sql, {'order_id': 1}):
        print(row)
# В каждой строке: order_id, товар, ресурс, норма, цена и сумма компонента.

```

Расчёт для одной строки (пример: евровинт):

2 шт. × 0,012 на изделие × 432,083333 руб. = 10,37 руб.
Итого по восьми компонентам = 11 448,12 руб.

Для заказа покупателя №1 контрольная строка запроса: (order_id=1, total_manufacturing_cost=11448.12, required_cost_positions=8, priced_positions=8). В машинное поле ответа вводите 11448,12.

Задание 4. Авторизация и информационная система

Шаг 1. Создать SQLite-таблицу users с уникальным логином, ролью Администратор/Пользователь, солью и PBKDF2-HMAC-SHA256-хешем пароля, is_blocked и failed_attempts.

Шаг 2. На форме входа сделать обязательные поля логина и пароля. Капча — пазл 3×3 из приложенных изображений. Игрок переставляет фрагменты мышью или кликами; сервер проверяет одноразовый challenge и правильный порядок.

Шаг 3. При ошибке пазла или пароля увеличить общий счётчик последовательных ошибок. На третьей подряд ошибке заблокировать пользователя и показать установленное заданием сообщение. При успешном входе сбросить счётчик.

Шаг 4. Создать рабочий стол после входа и вывести заданное сообщение об успешной авторизации. Сессию хранить в HttpOnly/SameSite cookie.

Шаг 5. Закрыть административный маршрут от роли Пользователь. На панели администратора добавить/редактировать логин, роль, пароль и статус блокировки; при добавлении проверять уникальность логина.

Шаг 6. Запуск: python3 solutions/task4_auth_api/app.py. Демо-входы: admin / Admin123! и user / User123!. Для промышленной эксплуатации необходимы HTTPS, CSRF-защита и постоянное хранилище сессий.

Сценарий	Ожидаемый результат
Верный вход	Рабочий стол; сообщение об успехе; сброс счётчика
Неверная капча/пароль	Ошибка и увеличение failed_attempts
Третья ошибка	Блокировка аккаунта
Администратор	Добавление/редактирование/разблокировка

Демо-учётные записи: admin / Admin123! и user / User123!. Смените их перед эксплуатацией; приложение учебное, без HTTPS/CSRF-защиты.

Как выполнить на практике

Действие 1. Запустите приложение из корня репозитория; оно само создаст exam_app.sqlite3, таблицы users/notes и две демонстрационные учётные записи. Откройте страницу /login в браузере. Для изолированного теста укажите EXAM_DB_PATH=/tmp/exam_auth_test.sqlite3, чтобы не менять обычную демонстрационную базу.

Действие 2. На странице заполните оба обязательных поля и соберите изображение: элементы 3×3 переставляются перетаскиванием или кликами по двум фрагментам. Затем войдите как admin / Admin123! либо user / User123!. Капча имеет случайное начальное расположение и одноразовый серверный challenge; готовый порядок из браузера принимается только если совпал с 0...8.

Действие 3. Проверьте отрицательный сценарий: три раза подряд введите неправильный пароль, каждый раз решив новый пазл правильно. После третьей попытки должна появиться фраза «Вы заблокированы. Обратитесь к администратору». Та же серия ошибок учитывает неверную капчу. Успешный вход сбрасывает failed_attempts.

Действие 4. Для проверки администрирования войдите под admin, откройте «Управление пользователями», добавьте логин с паролем и ролью. Повторите добавление того же логина — приложение должно показать сообщение о дубликате. В таблице пользователей поменяйте роль/пароль или снимите is_blocked; при разблокировке счётчик ошибок сбрасывается.

Действие 5. Для самопроверки выйдите из браузерного сценария и запустите unittest. Тесты запускают сервер на временном порту и отдельной временной БД, поэтому демо-базу не портят.

Запуск и адрес страницы входа:

```

python3 solutions/task4_auth_api/app.py
# открыть в браузере: http://127.0.0.1:8000/login
# тестовые пары: admin / Admin123!; user / User123!

```

Чтобы тестировать блокировку в отдельной БД и не портить демо-вход:

```

rm -f /tmp/exam_auth_test.sqlite3
EXAM_DB_PATH=/tmp/exam_auth_test.sqlite3 APP_PORT=8001 python3 solutions/task4_auth_api/app.py
# открыть http://127.0.0.1:8001/login

```

Проверить все сценарии задания 4–5:

```
python3 -m unittest discover -s solutions/task4_auth_api/tests -v
```

Успешный вход показывает «Вы успешно авторизовались». Пользователь видит свой рабочий стол; маршрут /admin/users для роли Пользователь закрыт ответом 403. Пароль в SQLite хранится как PBKDF2-HMAC-SHA256 с

солью, cookie сессии помечена HttpOnly и SameSite.

Задание 5. API заметок и тестовая коллекция

Шаг 1. В той же SQLite-БД создать notes: id, title, content, user_id (FK users), created_at. При первом запуске добавить тестовые записи.

Шаг 2. Реализовать GET /notes и алиас GET /api/notes. Соединить notes с users: title_user = title + ' - ' + login; content передать без изменения; created_at форматировать ДД.ММ.ГГГГ.

Шаг 3. Поддерживать limit от 1 до 1000 и положительный user_id. Неизвестные, повторные, пустые/неверно типизированные параметры возвращают JSON 400.

Шаг 4. Пустая выборка — корректный 200 с []. Сбой БД — 500 с JSON-полем error. Неподдерживаемый метод — 405, Allow: GET. Успех и ошибки API всегда имеют application/json.

Шаг 5. Импортировать Exam_API.postman_collection.json и Exam_API.postman_environment.json. Обычный сервер слушает 8000; тестовый сбой для 500 включается только флагами EXAM_TEST_MODE=1 и EXAM_FORCE_DB_ERROR=1 на отдельном экземпляре 8001.

Шаг 6. Запустить: python3 -m unittest discover -s solutions/task4_auth_api/tests -v. Проверяются 200, 400, 500, JSON, [], 405, блокировка и админские операции; результат — 7 тестов пройдены.

Код	Сценарий	Ответ
200	Штатно / данных нет	JSON-массив / []
400	Некорректный параметр	JSON с error
500	Сбой БД	JSON с error
405	Метод не GET	JSON и Allow: GET

```
[{"id":5,"title_user":"Итоги - admin","content":"Сохранить результаты тестирования.","formatted_date":"05.10.2026"}]
```

Коллекция Postman и environment в task5_api/. При тестовом 500 запускается второй экземпляр с EXAM_TEST_MODE=1 и EXAM_FORCE_DB_ERROR=1.

Как выполнить на практике

Действие 1. При старте приложения задание 4 создаёт общую БД, таблицу notes и пять тестовых заметок. Основной маршрут — GET /notes, второй адрес — GET /api/notes. Клиент получает JSON; таблицу notes соединяйте с users по notes.user_id = users.id.

Действие 2. Отправьте запрос без фильтров, затем с limit=2. Для каждого ответа проверьте преобразование: title_user = title + пробел-дефис-пробел + login; content не меняйте; дату YYYY-MM-DD преобразуйте в DD.MM.YYYY. Параметр limit допустим от 1 до 1000, user_id — положительное целое; неизвестный, повторный или неправильный параметр — 400.

Действие 3. Повторите GET с несуществующим user_id: это не ошибка сервера, а 200 и пустой массив []. Отправьте POST /notes: ожидаются 405 и заголовок Allow: GET. Все три маршрута ошибок/успеха должны иметь application/json и тело, пригодное для json.loads.

Действие 4. Импортируйте в Postman оба файла из task5_api: коллекцию и environment. Обычные проверки 200/400 направьте на порт 8000. Для 500 запустите отдельный процесс с двумя тестовыми флагами и портом 8001, затем запустите третий запрос коллекции.

Проверить обычный JSON-ответ и обработку неверного параметра:

```
curl -i 'http://127.0.0.1:8000/notes?limit=2'
curl -i 'http://127.0.0.1:8000/notes?limit=abc'
```

Тестовый сервер для проверки ответа 500 (Linux/macOS):

```
EXAM_TEST_MODE=1 EXAM_FORCE_DB_ERROR=1 APP_PORT=8001 \
EXAM_DB_PATH=/tmp/exam_api_fault.sqlite3 \
python3 solutions/task4_auth_api/app.py
# затем: curl -i 'http://127.0.0.1:8001/notes'
```

Форма одного объекта, которую должен проверить клиент:

```
{
  "id": 5,
  "title_user": "Итоги - admin",
  "content": "Сохранить результаты тестирования.",
  "formatted_date": "05.10.2026"
}
```

Ожидайте: GET /notes — 200 и JSON-массив; limit=abc — 400 и {"error": ...}; пустая выборка — 200 и []; включённый тестовый сбой на 8001 — 500 и JSON с полем error; POST /notes — 405, Allow: GET.

Задание 6. Документация API

Шаг 1. На базе исходного DOCX-шаблона заполнить базовый URL, GET /notes и алиас, параметры, назначение метода и формат ответа.

Шаг 2. Описать статусы 200, 400, 500 и фактически реализованный 405, включая смысл пустого массива при отсутствии заметок.

Шаг 3. Привести примеры успешного запроса и ответов на неверный параметр и сбой БД; указать формат даты и структуру JSON-объекта.

Готовый документ: task6_docs/API_документация.docx; воспроизводимый генератор — task6_docs/build_docx.py.

Документ task6_docs/API_документация.docx заполнен по исходному шаблону; генератор — task6_docs/build_docx.py.

Как выполнить на практике

Действие 1. Сначала откройте Прил_6_ОЗ_КИМ_09.02.07-5-2027.docx и посмотрите таблицы шаблона. В первой таблице замените значения базового URL и строки метода; в её колонках укажите название метода, query-параметры, назначение, поля/формат ответа. Во второй таблице перечислите коды ответа и поведение для каждого.

Действие 2. Для этого решения внесите GET /notes и алиас GET /api/notes; распишите limit и user_id с границами; укажите точное правило title_user, неизменяемый content и дату ДД.ММ.ГГГГ. Отдельными строками добавьте 200, 400, 500 и 405 (Allow: GET), включая кейс 200 + [] для пустого результата.

Действие 3. Добавьте примеры запроса и тела ответа: один успешный массив, один ответ 400, один ответ 500. В примерах используйте те же имена полей и формат даты, что реально выдаёт app.py. Не придумывайте методы, которых приложение не реализует.

Действие 4. Для воспроизводимого заполнения из корня запустите build_docx.py: он открывает исходный шаблон, заполняет таблицы 0 и 1, добавляет примеры и сохраняет итог в task6_docs/API_документация.docx. Откройте итоговый DOCX и визуально проверьте, что таблицы не обрезаны.

Пересобрать заполненную проектную документацию:

```
python3 solutions/task6_docs/build_docx.py
```

Проверочный пример успешного запроса:

```
GET http://127.0.0.1:8000/notes?limit=2
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
```

```
[{"id":5,"title_user":"Итоги - admin","content":"Сохранить результаты тестирования.","formatted_date":"05.10.2026"}]
```

Итоговый файл — solutions/task6_docs/API_документация.docx. Сверьте каждое указанное поле с ответом GET /notes и каждую ветку статуса с тестами test_app.py.

Задание 7. Очистка данных и временной ряд

Шаг 1. Прочитать dataset_2026.xlsx; сохранить исходный порядок и все 30 строк. Удалять разрешено только полные дубликаты; в данном наборе их нет.

Шаг 2. Стандартизировать transaction_id, пробелы/регистр товаров и менеджеров. Исправить известную опечатку Пылесос Dison на Пылесос Dyson.

Шаг 3. Преобразовать значения количества '3 шт.' в число 3, цены '294980 руб.' — в 294980. Проверить даты на принадлежность 2026 году.

Шаг 4. Нормализовать категории по типу товара; исправить латинские символы-двойники в категории; заполнить пропущенную категорию по названию продукта.

Шаг 5. Выручка транзакции = quantity × price. Суммировать все транзакции по календарным месяцам без фильтрации магазинов, менеджеров и категорий. Для долей использовать общую годовую выручку.

Шаг 6. Построить линейный график января-декабря и добавить горизонтальную линию среднего арифметического 12 месячных значений. Результат: clean_dataset_2026.xlsx; анализ/график: analysis_2026.xlsx.

Шаг 7. Контроль: 30 строк; удалено 0; полных дублей 0; журнал содержит 47 исправлений в 27 строках. Годовая выручка 7759680, среднее 646640, максимум Ноябрь 1794540, минимум Сентябрь 127949.

Месячная выручка и доли

Месяц	Выручка, руб.	Доля, %
Январь	413985	5,34
Февраль	1278414	16,48
Март	1208502	15,57
Апрель	526072	6,78
Май	279840	3,61
Июнь	598556	7,71
Июль	234172	3,02
Август	332508	4,29
Сентябрь	127949	1,65
Октябрь	716678	9,24
Ноябрь	1794540	23,13
Декабрь	248464	3,20
Итого	7759680	100,00
Контрольный ответ		Значение
Средняя за 12 месяцев		646640 руб.
Максимум		Ноябрь — 1794540 руб.

Контрольный ответ	Значение
Минимум	Сентябрь — 127949 руб.
Размах	1666591 руб.
Линия среднего графика 7.9	646640 руб.

Выручка волатильна, устойчивого непрерывного роста нет. У TXN-0001 цена утюга Philips равна 897270 руб./шт.; значение отмечено как выброс, но сохранено, поскольку исправленная цена и правило её подстановки в ТЗ отсутствуют. Этот ряд формирует ноябрьский пик.

В исходных материалах отсутствуют Analytics_autotest и конкретные тексты вопросов 7.2–7.9. Вместо неподтверждённой имитации добавлен preflight_check.py: он проверяет явные критерии очистки, итоговые числа и график. Это локальная дополнительная проверка, не официальный тестер; контрольный код 7.1 можно получить только в предоставленном экзаменационном приложении. SHA-256 не является его заменой.

Как выполнить на практике

Действие 1. Сделайте рабочую копию dataset_2026.xlsx и проверьте заголовки A:H: transaction_id, date, product, category, quantity, price, shop, manager. В исходнике 30 записей и три пустых дополнительных столбца; очищенный файл должен сохранить все 30 записей в том же порядке. Удаляйте только полное совпадение всех полей — здесь полных дубликатов нет.

Действие 2. Очистите ID от пробелов/регистра и приведите к TXN-0001. Нормализуйте товар: trim пробелов, единый регистр и исправление Dison -> Dyson. Количество очистите от суффикса «шт.», цену — от «руб.», затем преобразуйте обе колонки в числа. Дату задайте датой Excel и проверьте, что все годы — 2026.

Действие 3. Пересчитайте категорию по названию товара, а не по грязному значению источника: пылесосы, телевизоры, стиральные машины, холодильники; утюг/чайник/микроволновая печь — «Мелкая бытовая техника». Это исправляет категории, случайно назначенные не тому товару, латинские буквы-двойники и пропуск. Нормализуйте пробелы в shop и единый формат инициалов manager.

Действие 4. Добавьте столбец revenue и для каждой строки вычислите quantity × price. Создайте таблицу из 12 календарных месяцев именно датами первого числа месяца, просуммируйте revenue по границам месяца; не фильтруйте категории, магазины и менеджеров.

Действие 5. Посчитайте годовой итог, долю месяца от годового итога, среднее 12 месячных значений, максимум/минимум и изменение к предыдущему месяцу. Постройте линейный график с X = январь...декабрь и Y = выручка; добавьте вторую серию с одинаковым средним значением для каждого из 12 месяцев и задайте ей горизонтальный пунктир.

Действие 6. Проще воспроизвести всё без ручной порчи числовых форматов командой clean_data.py. Откройте созданные clean_dataset_2026.xlsx и analysis_2026.xlsx, проверьте листы Данные, Месячная выручка, Итоги и Журнал очистки. Затем запустите добавленную локальную предварительную проверку: она сверяет каждый критерий, который можно вывести из приложенного ТЗ. Официальный Analytics_autotest в материалах отсутствует, поэтому локальная проверка не выдаёт и не имитирует его контрольный код.

Формулы для ручного расчёта в русской версии Excel (если лист Данные уже очищен):

```
I2: =E2*F2
B2: =СУММЕСЛИМН(Данные!$I$2:$I$31;Данные!$B$2:$B$31;">="&A2;Данные!$B$2:$B$31;"<"&DATA(ГОД(A2);МЕСЯЦ(A2)+1;1))
B14: =СУММ(B2:B13)
C2: =B2/$B$14*100
D3: =B3-B2
E3: =D3/B2*100
F2: =СРЗНАЧ($B$2:$B$13)
# A2:A13 — даты 01.01.2026 ... 01.12.2026. Протяните формулы вниз; B14 — годовой итог.
```

Автоматически очистить и построить две итоговые книги:

```
python3 solutions/task7_analysis/clean_data.py
```

Проверить все явные требования задания 7 локальным preflight-check (это не официальный тестер):

```
python3 solutions/task7_analysis/preflight_check.py
```

После выполнения: 30 строк, удалено 0, полных дубликатов 0; годовая выручка 7759680, средняя месячная 646640; максимум — ноябрь 1794540, минимум — сентябрь 127949. TXN-0001 (утюг Philips, 897270 руб./шт.) отмечен как ценовой выброс, но не заменён без правила из исходного ТЗ. Локальный preflight-check проверяет явные требования, но не выдаёт официальный код 7.1.

Полные текстовые листинги

Следующие приложения включают полный исходный текст файлов проекта. Чтобы запустить решение, используйте соответствующие команды из solutions/README_RU.md.

Приложение А. Исходная ER-модель

Файл: solutions/task1_er/ER-модель.mmd

```
%% Задание 1. Логическая ER-модель в ЗНФ.
%% Цена/скидка ссылается ровно на одну сущность: товар ИЛИ ресурс.
erDiagram
    CUSTOMERS ||--o{ CUSTOMER_ORDERS : оформляет
    CUSTOMER_ORDERS ||--|{ ORDER_ITEMS : содержит
    PRODUCTS ||--o{ ORDER_ITEMS : заказан

    PRODUCTS ||--o{ PRODUCT_SPECIFICATION : состоит_из
    RESOURCES ||--o{ PRODUCT_SPECIFICATION : входит_в

    PRODUCTS o|--o{ PRICES : цена_товара
    RESOURCES o|--o{ PRICES : цена_ресурса
    PRODUCTS o|--o{ DISCOUNTS : скидка_товара
    RESOURCES o|--o{ DISCOUNTS : скидка_ресурса

    PRODUCTION_ORDERS ||--|{ PRODUCTION_ORDER_ITEMS : содержит
    PRODUCTS ||--o{ PRODUCTION_ORDER_ITEMS : выпускается

    CUSTOMERS {
        string customer_id PK
        string name
        string inn
        string address
        string phone
        string party_type
    }
    CUSTOMER_ORDERS {
        int order_id PK
        string customer_id FK
        date order_date
        string status
    }
    ORDER_ITEMS {
        int order_item_id PK
        int source_order_item_id
        int order_id FK
        int product_id FK
        int quantity
        decimal unit_price_at_order "nullable snapshot"
        decimal discount_per_unit
    }
    PRODUCTS {
        int product_id PK
        int source_product_id
        string name
        string category
        boolean is_placeholder
    }
    RESOURCES {
        int resource_id PK
        int source_resource_id
        string name
        string resource_type
    }
    PRODUCT_SPECIFICATION {
        int specification_id PK
        int product_id FK
        int resource_id FK
        decimal quantity_per_product
    }
    PRICES {
        int price_id PK
        int product_id FK "nullable"
        int resource_id FK "nullable"
        decimal unit_price
    }
    DISCOUNTS {
        int discount_id PK
        int source_discount_id
        int product_id FK "nullable"
        int resource_id FK "nullable"
        decimal discount_percent
    }
    PRODUCTION_ORDERS {
        int production_order_id PK
        string order_number
        date launch_date
        string department
    }
    PRODUCTION_ORDER_ITEMS {
        int production_order_item_id PK
        int production_order_id FK
        int product_id FK
        decimal quantity
        string unit
    }
```

Приложение В. Генератор PDF ER-диаграммы

Файл: solutions/task1_er/generate_diagram.py

```
#!/usr/bin/env python3
"""Генератор PDF ER-диаграммы (нужен reportlab)."""
from pathlib import Path
from math import hypot

from reportlab.lib.colors import HexColor, white
from reportlab.lib.pagesizes import landscape, A3
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.pdfgen import canvas

HERE = Path(__file__).resolve().parent
OUTPUT = HERE / "ER_диаграмма.pdf"
FONT = Path("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf")
FONT_BOLD = Path("/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf")
pdfmetrics.registerFont(TTFont("DejaVu", str(FONT)))
pdfmetrics.registerFont(TTFont("DejaVu-Bold", str(FONT_BOLD)))

PAGE_W, PAGE_H = landscape(A3)
NAVY = HexColor("#172B4D")
BLUE = HexColor("#4058D6")
MINT = HexColor("#DDF4EE")
PALE = HexColor("#F2F5FB")
MUTED = HexColor("#52627A")
EDGE = HexColor("#58708E")

entities = [
    {"key": "customers", "ru": "Заказчики", "row": 0, "col": 0,
     "fields": [{"PK", "customer_id TEXT"}, ("", "name"), ("", "inn"), ("", "address"), ("", "phone"), ("",
     "party_type")]},
    {"key": "customer orders", "ru": "Заказы покупателей", "row": 0, "col": 1,
     "fields": [{"PK", "order_id"}, ("FK", "customer_id"), ("", "order_date"), ("", "status")]},
    {"key": "order items", "ru": "Позиции заказа", "row": 0, "col": 2,
     "fields": [{"PK", "order_item_id"}, ("", "source_order_item_id"), ("FK", "order_id"), ("FK", "product_id"), ("",
     "quantity > 0"), ("", "unit_price_at_order", "discount_per_unit")]},
    {"key": "products", "ru": "Товары", "row": 0, "col": 3,
     "fields": [{"PK", "product_id"}, ("", "source_product_id"), ("", "name"), ("", "category"), ("",
     "is_placeholder")]},
    {"key": "resources", "ru": "Материалы и операции", "row": 0, "col": 4,
     "fields": [{"PK", "resource_id"}, ("", "source_resource_id"), ("", "name"), ("", "resource_type")]},
    {"key": "production orders", "ru": "Заказы на производство", "row": 1, "col": 0,
     "fields": [{"PK", "production_order_id"}, ("", "order_number"), ("", "launch_date"), ("", "department")]},
    {"key": "production order items", "ru": "Выпускаемая продукция", "row": 1, "col": 1,
     "fields": [{"PK", "production_order_item_id"}, ("FK", "production_order_id"), ("FK", "product_id"), ("", "quantity",
     "unit")]},
    {"key": "product specification", "ru": "Спецификация / BOM", "row": 1, "col": 2,
     "fields": [{"PK", "specification_id"}, ("FK", "product_id"), ("FK", "resource_id"), ("", "quantity_per_product")]},
    {"key": "prices", "ru": "Цены", "row": 1, "col": 3,
     "fields": [{"PK", "price_id"}, ("FK", "product_id NULL"), ("FK", "resource_id NULL"), ("", "unit_price >= 0)]},
    {"key": "discounts", "ru": "Скидки", "row": 1, "col": 4,
     "fields": [{"PK", "discount_id"}, ("", "source_discount_id"), ("FK", "product_id NULL"), ("FK", "resource_id
     NULL"), ("", "discount_percent")]},
    ]

MARGIN_X = 26
GAP_X = 12
CARD_W = (PAGE_W - 2*MARGIN_X - 4*GAP_X) / 5
CARD_H = 160
ROW_Y = {0: 590, 1: 333}

def wrap_text(text: str, font_name: str, size: float, max_width: float) -> list[str]:
    words = text.split()
    lines, current = [], ""
    for word in words:
        candidate = f"{current} {word}".strip()
        if current and pdfmetrics.stringWidth(candidate, font_name, size) > max_width:
            lines.append(current)
            current = word
        else:
            current = candidate
    if current:
        lines.append(current)
    return lines

def draw_card(c: canvas.Canvas, entity: dict) -> dict:
    x = MARGIN_X + entity["col"] * (CARD_W + GAP_X)
    y = ROW_Y[entity["row"]]
    c.setFillColor(white)
    c.setStrokeColor(HexColor("#C9D3E2"))
    c.setLineWidth(1.0)
    c.roundRect(x, y, CARD_W, CARD_H, 9, fill=1, stroke=1)
    c.setFillColor(NAVY if entity["row"] == 0 else HexColor("#234E63"))
    c.roundRect(x, y + CARD_H - 35, CARD_W, 35, 9, fill=1, stroke=0)
    c.rect(x, y + CARD_H - 35, CARD_W, 9, fill=1, stroke=0)
    c.setFillColor(white)
    c.setFont("DejaVu-Bold", 9.5)
    c.drawString(x + 9, y + CARD_H - 15, entity["key"])
    c.setFont("DejaVu", 7.5)
    c.drawString(x + 9, y + CARD_H - 28, entity["ru"])
    line_y = y + CARD_H - 49
    for tag, value in entity["fields"]:
        prefix = (tag + " ") if tag else ""
        color = BLUE if tag.startswith("PK") else (HexColor("#17735F") if tag.startswith("FK") else NAVY)
        c.setFillColor(color)
        c.setFont("DejaVu-Bold" if tag else "DejaVu", 7.8)
        for line in wrap_text(prefix + value, "DejaVu-Bold" if tag else "DejaVu", 7.8, CARD_W - 17):
            c.drawString(x + 9, line_y, line)
            line_y -= 11.7
    return {"x": x, "y": y, "w": CARD_W, "h": CARD_H, "cx": x + CARD_W / 2, "cy": y + CARD_H / 2}

def connect(c: canvas.Canvas, start: tuple[float, float], end: tuple[float, float], label: str,
           dashed: bool=False, label_offset: tuple[float, float]=(0, 6)) -> None:
    x1, y1 = start; x2, y2 = end
```

```

c.saveState()
c.setStrokeColor(EDGE)
c.setFill(EDGE)
c.setLineWidth(1.1)
if dashed:
    c.setDash(3, 2)
c.line(x1,y1,x2,y2)
length = hypot(x2-x1,y2-y1) or 1
ux,uy=(x2-x1)/length,(y2-y1)/length
size=5
bx,by=x2-ux*size,y2-uy*size
px,py=-uy*size*.55,ux*size*.55
path=c.beginPath(); path.moveTo(x2,y2); path.lineTo(bx+px,by+py); path.lineTo(bx-px,by-py); path.close()
c.drawPath(path,fill=1,stroke=0)
c.setDash()
mx,my=(x1+x2)/2+label_offset[0],(y1+y2)/2+label_offset[1]
text_w=pdfmetrics.stringWidth(label,"DejaVu-Bold",7.1)
c.setFill(white)
c.roundRect(mx-text_w/2-3,my-2,text_w+6,11,3,fill=1,stroke=0)
c.setFill(BLUE)
c.setFont("DejaVu-Bold",7.1)
c.drawCentredString(mx,my,label)
c.restoreState()

def main() -> None:
    c = canvas.Canvas(str(OUTPUT), pagesize=(PAGE_W,PAGE_H))
    c.setTitle("ER-диаграмма – экзамен 09.02.07")
    c.setAuthor("Экзаменационные материалы")
    c.setFill(NAVY); c.setFont("DejaVu-Bold",18)
    c.drawString(26,PAGE_H-31,"Задание 1. ER-диаграмма информационной системы производства")
    c.setFill(MUTED); c.setFont("DejaVu",9)
    c.drawString(26,PAGE_H-49,"Логическая модель в третьей нормальной форме; PK – первичный ключ, FK – внешний ключ, FK* – один из двух взаимоисключающих FK.")

    boxes={entity["key"]:draw_card(c,entity) for entity in entities}
    b=boxes
    # Основной контур клиентского заказа.
    connect(c,(b["customers"])[x]+b["customers"]w,b["customers"]cy+18),(b["customer_orders"]x,b["customer_orders"]cy+18),1:N)
    connect(c,(b["customer_orders"]x)+b["customer_orders"]w,b["customer_orders"]cy+18),(b["order_items"]x,b["order_items"]cy+18),1:N)
    connect(c,(b["products"]x)+b["products"]cy-18),(b["order_items"]x)+b["order_items"]w,b["order_items"]cy-18,1:N,label_offset=(0,-11))
    # Состав изделия: связь M:N через таблицу спецификации.
    connect(c,(b["products"]cx)+b["products"]y),(b["product_specification"]x)+b["product_specification"]w,1:N)
    connect(c,(b["resources"]cx)+b["resources"]y),(b["product_specification"]x)+b["product_specification"]w-24,b["product_specification"]y)+b["product_specification"]h,1:N,label_offset=(0,-10))
    # Цены и скидки опционально относятся либо к товару, либо к ресурсу.
    connect(c,(b["products"]cx)+b["products"]y),(b["prices"]cx)+b["prices"]y)+b["prices"]h,1:0..N,
    True)
    connect(c,(b["resources"]cx)+b["resources"]w-22,b["resources"]y),(b["prices"]x)+b["prices"]w,b["prices"]y)+b["prices"]h-25,1:0..N,True,(4,-6))
    connect(c,(b["products"]x)+b["products"]w,b["products"]y)+20,(b["discounts"]x)+b["discounts"]y)+b["discounts"]h-20,1:0..N,True,(0,9))
    connect(c,(b["resources"]cx)+b["resources"]y),(b["discounts"]cx)+b["discounts"]y)+b["discounts"]h,1:0..N,True,(0,9))
    # Производственный заказ.
    connect(c,(b["production_orders"]x)+b["production_orders"]w,b["production_orders"]cy),(b["production_order_items"]x)+b["production_order_items"]cy,1:N)
    connect(c,(b["products"]x)+b["products"]y)+25,(b["production_order_items"]x)+b["production_order_items"]w,b["production_order_items"]y)+b["production_order_items"]h,1:N,label_offset=(0,-10))

    # Легенда и правила ссылочной целостности.
    note_y=282
    c.setFill(NAVY); c.setFont("DejaVu-Bold",10)
    c.drawString(26,note_y,"Связи и ограничения")
    c.setFill(MUTED); c.setFont("DejaVu",8.2)
    left_notes=[
        "Заказчик 1:N заказов; заказ 1:N позиций; товар 1:N позиций.",
        "Товар M:N ресурс через product_specification; количество – норма на единицу продукции.",
        "Производственный заказ 1:N строк выпуска; строка указывает товар, количество и единицу измерения.",
    ]
    right_notes=[
        "resource_type различает «Материал» и «Операция»; обе сущности тарифицируются одинаково.",
        "B prices и discounts ровно один FK: product_id ИЛИ resource_id (CHECK в schema.sql).",
        "Surrogate PK сохраняют строки с повторными исходными ID; пропущенные товары помечены is_placeholder=1.",
    ]
    for i,text in enumerate(left_notes):
        c.drawString(26,note_y-16-i*15,"• " + text)
    for i,text in enumerate(right_notes):
        c.drawString(PAGE_W/2+4,note_y-16-i*15,"• " + text)
    c.setFill(HexColor("#7A879A")); c.setFont("DejaVu",7)
    c.drawString(26,27,"Файлы реализации: schema.sql и seed.py. Модель не хранит вычисляемую себестоимость – она считается запросом по нормам спецификации и ценам ресурсов.")
    c.showPage(); c.save()
    print(OUTPUT)

if __name__ == "__main__":
    main()

```

Приложение С. SQL-схема

Файл: solutions/task2_database/schema.sql

```
-- Задание 2. Схема SQLite (3НФ). Все внешние ключи включаются в seed.py.
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS customers (
  customer_id TEXT PRIMARY KEY,
  name        TEXT NOT NULL,
  inn         TEXT NOT NULL DEFAULT '',
  address     TEXT NOT NULL DEFAULT '',
  phone       TEXT NOT NULL DEFAULT '',
  party_type  TEXT NOT NULL CHECK (party_type IN ('Покупатель', 'Поставщик', 'Прочее'))
);

CREATE TABLE IF NOT EXISTS products (
  product_id    INTEGER PRIMARY KEY,
  source_product_id INTEGER NOT NULL,
  name          TEXT NOT NULL,
  category      TEXT NOT NULL DEFAULT '',
  is_placeholder INTEGER NOT NULL DEFAULT 0 CHECK (is_placeholder IN (0, 1))
);
CREATE INDEX IF NOT EXISTS idx_products_source_id ON products(source_product_id);

-- В исходном файле «Материалы.json» есть как материалы, так и операции.
-- Операции хранятся в том же справочнике, различие задаёт resource_type.
CREATE TABLE IF NOT EXISTS resources (
  resource_id    INTEGER PRIMARY KEY,
  source_resource_id INTEGER NOT NULL,
  name          TEXT NOT NULL,
  resource_type  TEXT NOT NULL CHECK (resource_type IN ('Материал', 'Операция'))
);
CREATE INDEX IF NOT EXISTS idx_resources_source_id ON resources(source_resource_id);

CREATE TABLE IF NOT EXISTS product_specification (
  specification_id INTEGER PRIMARY KEY,
  product_id       INTEGER NOT NULL REFERENCES products(product_id),
  resource_id       INTEGER NOT NULL REFERENCES resources(resource_id),
  quantity_per_product NUMERIC NOT NULL CHECK (quantity_per_product > 0),
  UNIQUE (product_id, resource_id)
);

CREATE TABLE IF NOT EXISTS customer_orders (
  order_id    INTEGER PRIMARY KEY,
  customer_id TEXT NOT NULL REFERENCES customers(customer_id),
  order_date  TEXT NOT NULL,
  status      TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS order_items (
  order_item_id    INTEGER PRIMARY KEY,
  source_order_item_id INTEGER NOT NULL,
  order_id         INTEGER NOT NULL REFERENCES customer_orders(order_id) ON DELETE CASCADE,
  product_id       INTEGER NOT NULL REFERENCES products(product_id),
  quantity         INTEGER NOT NULL CHECK (quantity > 0),
  unit_price_at_order NUMERIC CHECK (unit_price_at_order IS NULL OR unit_price_at_order >= 0),
  discount_per_unit  NUMERIC NOT NULL DEFAULT 0 CHECK (discount_per_unit >= 0),
  CHECK (unit_price_at_order IS NULL OR discount_per_unit <= unit_price_at_order)
);

-- В Excel-приложениях есть заказ на производство. Эти таблицы входят в модель,
-- даже если JSON-файлы для них отдельно не приложены.
CREATE TABLE IF NOT EXISTS production_orders (
  production_order_id INTEGER PRIMARY KEY,
  order_number        TEXT NOT NULL,
  launch_date         TEXT NOT NULL,
  department           TEXT NOT NULL DEFAULT ''
);

CREATE TABLE IF NOT EXISTS production_order_items (
  production_order_item_id INTEGER PRIMARY KEY,
  production_order_id      INTEGER NOT NULL REFERENCES production_orders(production_order_id) ON DELETE CASCADE,
  product_id               INTEGER NOT NULL REFERENCES products(product_id),
  quantity                 NUMERIC NOT NULL CHECK (quantity > 0),
  unit                     TEXT NOT NULL DEFAULT 'шт'
);

-- Полиморфная ссылка из JSON разложена на два nullable FK.
-- CHECK гарантирует, что цена относится ровно к одному товару ИЛИ ресурсу.
CREATE TABLE IF NOT EXISTS prices (
  price_id    INTEGER PRIMARY KEY,
  product_id  INTEGER REFERENCES products(product_id),
  resource_id INTEGER REFERENCES resources(resource_id),
  unit_price  NUMERIC NOT NULL CHECK (unit_price >= 0),
  CHECK (
    (product_id IS NOT NULL AND resource_id IS NULL) OR
    (product_id IS NULL AND resource_id IS NOT NULL)
  )
);

CREATE TABLE IF NOT EXISTS discounts (
  discount_id    INTEGER PRIMARY KEY,
  source_discount_id INTEGER NOT NULL,
  product_id     INTEGER REFERENCES products(product_id),
  resource_id     INTEGER REFERENCES resources(resource_id),
  discount_percent NUMERIC NOT NULL CHECK (discount_percent BETWEEN 0 AND 100),
  CHECK (
    (product_id IS NOT NULL AND resource_id IS NULL) OR
    (product_id IS NULL AND resource_id IS NOT NULL)
  )
);

CREATE INDEX IF NOT EXISTS idx_orders_customer ON customer_orders(customer_id);
CREATE INDEX IF NOT EXISTS idx_order_items_order ON order_items(order_id);
CREATE INDEX IF NOT EXISTS idx_order_items_product ON order_items(product_id);
CREATE INDEX IF NOT EXISTS idx_spec_product ON product_specification(product_id);
CREATE INDEX IF NOT EXISTS idx_spec_resource ON product_specification(resource_id);
CREATE INDEX IF NOT EXISTS idx_prices_resource ON prices(resource_id);
CREATE INDEX IF NOT EXISTS idx_prices_product ON prices(product_id);
```

Приложение D. Загрузчик JSON/SQLite

Файл: solutions/task2_database/seed.py

```
#!/usr/bin/env python3
"""Создаёт manufacturing.sqlite3 и загружает предоставленные JSON-данные.

Запуск из любой папки:
python3 solutions/task2_database/seed.py
Или укажите свои пути:
python3 seed.py --data-dir /path/to/source --db-path ./manufacturing.sqlite3
"""
from __future__ import annotations

import argparse
import json
import sqlite3
from collections import Counter
from pathlib import Path
from typing import Any

HERE = Path(__file__).resolve().parent
REPO_ROOT = HERE.parents[1]
SYNTHETIC_ID_BASE = 1_000_000_000

def read_records(path: Path) -> list[dict[str, Any]]:
    with path.open("r", encoding="utf-8-sig") as stream:
        value = json.load(stream)
        if isinstance(value, list):
            return value
        if isinstance(value, dict) and isinstance(value.get("records"), list):
            return value["records"]
        raise ValueError(f"Неожиданный формат JSON: {path.name}")

def allocate_surrogate(used_ids: set[int], counter: list[int]) -> int:
    while counter[0] in used_ids:
        counter[0] += 1
    result = counter[0]
    used_ids.add(result)
    counter[0] += 1
    return result

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data-dir", type=Path, default=REPO_ROOT,
                        help="папка с исходными JSON (по умолчанию корень проекта)")
    parser.add_argument("--db-path", type=Path, default=HERE / "manufacturing.sqlite3",
                        help="путь создаваемой базы SQLite")
    args = parser.parse_args()
    data_dir = args.data_dir.resolve()
    db_path = args.db_path.resolve()
    db_path.parent.mkdir(parents=True, exist_ok=True)

    source_names = [
        "Заказчики.json", "Заказы.json", "Позиции заказов.json", "Товары.json",
        "Материалы.json", "Спецификация.json", "Цены.json", "Скидки.json",
    ]
    missing_files = [name for name in source_names if not (data_dir / name).is_file()]
    if missing_files:
        raise FileNotFoundError("Не найдены исходные файлы: " + ", ".join(missing_files))

    data = {name: read_records(data_dir / name) for name in source_names}
    customers = data["Заказчики.json"]
    orders = data["Заказы.json"]
    order_items = data["Позиции заказов.json"]
    products = data["Товары.json"]
    resources = data["Материалы.json"]
    specification = data["Спецификация.json"]
    price_rows = data["Цены.json"]
    discount_rows = data["Скидки.json"]

    # Проверяем внешние ссылки до загрузки.
    known_product_source_ids = {int(row["id"]) for row in products}
    referenced_product_ids = {int(row["product_id"]) for row in order_items}
    referenced_product_ids.update(int(row["product_id"]) for row in specification)
    for rows in (price_rows, discount_rows):
        referenced_product_ids.update(
            int(row["item_id"]) for row in rows if row["item_type"] == "Товар"
        )
    missing_product_ids = sorted(referenced_product_ids - known_product_source_ids)

    # В источнике встречаются повторные ID:
    # products: 887; resources: 456; order items: 31 и 81; discounts: 15.
    # Для родительских справочников используем исходный ID для первой записи,
    # а последующим выдаём surrogate key и сохраняем source_*_id. Все внешние
    # ссылки на повторный ID связываются с первой записью; неоднозначность ниже
    # явно сообщается. Строки заказа и скидки получают собственные surrogate PK.
    duplicate_product_ids = sorted(
        source_id for source_id, count in Counter(int(row["id"]) for row in products).items()
        if count > 1
    )
    duplicate_resource_ids = sorted(
        source_id for source_id, count in Counter(int(row["id"]) for row in resources).items()
        if count > 1
    )
    duplicate_order_item_ids = sorted(
        source_id for source_id, count in Counter(int(row["id"]) for row in order_items).items()
        if count > 1
    )
    duplicate_discount_ids = sorted(
        source_id for source_id, count in Counter(int(row["id"]) for row in discount_rows).items()
        if count > 1
    )

    product_ref: dict[int, int] = {}
    used_product_ids: set[int] = set()
    surrogate_counter = [SYNTHETIC_ID_BASE]
```

```

product_values = []
for row in products:
    source_id = int(row["id"])
    internal_id = source_id if source_id not in used_product_ids else allocate_surrogate(used_product_ids,
surrogate_counter)
    used_product_ids.add(internal_id)
    product_ref.setdefault(source_id, internal_id)
    product_values.append((internal_id, source_id, row["name"], row.get("category", "") or "", 0))
for source_id in missing_product_ids:
    internal_id = source_id if source_id not in used_product_ids else allocate_surrogate(used_product_ids,
surrogate_counter)
    used_product_ids.add(internal_id)
    product_ref[source_id] = internal_id
    product_values.append((internal_id, source_id,
        f"Нет в исходном справочнике: product_id={source_id}", "", 1))

resource_ref: dict[int, int] = {}
typed_resource_ref: dict[tuple[int, str], int] = {}
used_resource_ids: set[int] = set()
resource_values = []
for row in resources:
    source_id = int(row["id"])
    resource_type = row["type"]
    internal_id = source_id if source_id not in used_resource_ids else allocate_surrogate(used_resource_ids,
surrogate_counter)
    used_resource_ids.add(internal_id)
    resource_ref.setdefault(source_id, internal_id)
    typed_resource_ref.setdefault((source_id, resource_type), internal_id)
    resource_values.append((internal_id, source_id, row["name"], resource_type))

schema = (HERE / "schema.sql").read_text(encoding="utf-8")
connection = sqlite3.connect(db_path)
connection.row_factory = sqlite3.Row
connection.execute("PRAGMA foreign_keys = ON")
try:
    connection.executescript(schema)
    with connection:
        # Перезагрузка идемпотентна: зависимости очищаются от дочерних к родительским.
        for table in (
            "prices", "discounts", "order_items", "production_order_items",
            "customer_orders", "product_specification", "production_orders",
            "resources", "products", "customers",
        ):
            connection.execute(f"DELETE FROM {table}")

        connection.executemany(
            """INSERT INTO customers
            (customer_id, name, inn, address, phone, party_type)
            VALUES (?, ?, ?, ?, ?, ?)""",
            [
                (
                    str(row["id"]), row.get("name", ""), row.get("inn", "") or "",
                    row.get("address", row.get("address", "")) or "",
                    row.get("phone", "") or "", row.get("type", "Прочее"),
                ) for row in customers],
        )
        connection.executemany(
            """INSERT INTO products
            (product_id, source_product_id, name, category, is_placeholder)
            VALUES (?, ?, ?, ?, ?)""",
            product_values,
        )
        connection.executemany(
            """INSERT INTO resources
            (resource_id, source_resource_id, name, resource_type)
            VALUES (?, ?, ?, ?)""",
            resource_values,
        )
        connection.executemany(
            """INSERT INTO product_specification
            (specification_id, product_id, resource_id, quantity_per_product)
            VALUES (?, ?, ?, ?)""",
            [
                (int(row["id"]), product_ref[int(row["product_id"])],
                 resource_ref[int(row["material_id"])], float(row["qty"]))
                for row in specification],
        )
        connection.executemany(
            """INSERT INTO customer_orders(order_id, customer_id, order_date, status)
            VALUES (?, ?, ?, ?)""",
            [
                (int(row["id"]), str(row["customer_id"]), row["order_date"], row["status"])
                for row in orders],
        )
        connection.executemany(
            """INSERT INTO order_items
            (order_item_id, source_order_item_id, order_id, product_id, quantity,
            unit_price_at_order, discount_per_unit)
            VALUES (?, ?, ?, ?, ?, ?, ?)""",
            [
                (internal_id, int(row["id"]), int(row["order_id"]), product_ref[int(row["product_id"])],
                 int(row["quantity"]), None, 0)
                for internal_id, row in enumerate(order_items, start=1)],
        )

    price_values = []
    for row in price_rows:
        if row["item_type"] == "Товар":
            product_id, resource_id = product_ref[int(row["item_id"])], None
        else:
            key = (int(row["item_id"]), row["item_type"])
            resource_id = typed_resource_ref.get(key, resource_ref.get(int(row["item_id"])))
            if resource_id is None:
                raise ValueError(f"Для цены не найден ресурс: {key}")
            product_id = None
            price_values.append((int(row["id"]), product_id, resource_id, float(row["price"])))
    connection.executemany(
        "INSERT INTO prices(price_id, product_id, resource_id, unit_price) VALUES (?, ?, ?, ?)",
        price_values,
    )

    discount_values = []
    for row in discount_rows:
        if row["item_type"] == "Товар":
            product_id, resource_id = product_ref[int(row["item_id"])], None

```

```

else:
    key = (int(row["item_id"]), row["item_type"])
    resource_id = typed_resource_ref.get(key, resource_ref.get(int(row["item_id"])))
    if resource_id is None:
        raise ValueError(f"Для скидки не найден ресурс: {key}")
    product_id = None
    discount_values.append((len(discount_values) + 1, int(row["id"]), product_id, resource_id,
                             float(row["discount_percent"])))
connection.executemany(
    """INSERT INTO discounts
    (discount_id, source_discount_id, product_id, resource_id, discount_percent)
    VALUES (?, ?, ?, ?, ?)""",
    discount_values,
)

violations = list(connection.execute("PRAGMA foreign_key_check"))
if violations:
    raise RuntimeError(f"Нарушены внешние ключи: {violations[5]}")
finally:
    connection.close()

print(f"База создана: {db_path}")
print("Загружено записей:")
for table in ("customers", "products", "resources", "product_specification",
              "customer_orders", "order_items", "prices", "discounts"):
    with sqlite3.connect(db_path) as check:
        count = check.execute(f"SELECT COUNT(*) FROM {table}").fetchone()[0]
    print(f" {table}: {count}")
if missing_product_ids:
    print("Добавлены помеченные заглушки для отсутствующих product_id:",
          ", ".join(map(str, missing_product_ids)))
if duplicate_product_ids:
    print("Повторные source_product_id (ссылки назначены первой записи):", duplicate_product_ids)
if duplicate_resource_ids:
    print("Повторные source_resource_id (FK товара/ресурса сохранены):", duplicate_resource_ids)
if duplicate_order_item_ids:
    print("Повторные source_order_item_id (строки сохранены с surrogate PK):", duplicate_order_item_ids)
if duplicate_discount_ids:
    print("Повторные source_discount_id (строки сохранены с surrogate PK):", duplicate_discount_ids)
print("Проверка PRAGMA foreign_key_check: ошибок нет.")

if __name__ == "__main__":
    main()

```


Приложение E. SQL-запросы задания 3

Файл: solutions/task3_query/order_cost.sql

```
-- Задание 3. Полная себестоимость клиентского заказа.
-- Параметр :order_id – номер заказа из customer_orders.
-- В product_specification находятся нормы расхода и материалов, и операций;
-- prices содержит цену ресурса. Скидки и продажная цена товара в себестоимость
-- производства не входят.
-- Если у любой строки заказа нет BOM или у любого ресурса нет цены,
-- итог не подменяется частичной суммой: total_manufacturing_cost будет NULL.

WITH components AS (
    SELECT
        o.order_id,
        oi.order_item_id,
        oi.product_id,
        oi.quantity AS ordered_quantity,
        ps.specification_id,
        ps.quantity_per_product,
        r.resource_id,
        p.price_id,
        p.unit_price
    FROM customer_orders AS o
    JOIN order_items AS oi
        ON oi.order_id = o.order_id
    LEFT JOIN product_specification AS ps
        ON ps.product_id = oi.product_id
    LEFT JOIN resources AS r
        ON r.resource_id = ps.resource_id
    LEFT JOIN prices AS p
        ON p.resource_id = r.resource_id
        AND p.product_id IS NULL
    WHERE o.order_id = :order_id
),
coverage AS (
    SELECT
        order_id,
        COUNT(DISTINCT order_item_id) AS order_positions,
        COUNT(DISTINCT CASE WHEN specification_id IS NOT NULL THEN order_item_id END)
            AS positions_with_bom,
        COUNT(specification_id) AS required_cost_positions,
        COUNT(price_id) AS priced_positions,
        SUM(ordered_quantity * quantity_per_product * unit_price) AS raw_total
    FROM components
    GROUP BY order_id
)
SELECT
    order_id,
    CASE
        WHEN positions_with_bom <> order_positions
            OR required_cost_positions = 0
            OR priced_positions <> required_cost_positions
        THEN NULL
        ELSE ROUND(raw_total, 2)
    END AS total_manufacturing_cost,
    required_cost_positions,
    priced_positions
FROM coverage;

-- Детализация того же расчёта: норма × цена × количество продукции.
SELECT
    o.order_id,
    oi.order_item_id,
    p.name AS product_name,
    oi.quantity AS ordered_quantity,
    r.name AS resource_name,
    r.resource_type,
    ps.quantity_per_product,
    pr.unit_price,
    ROUND(ps.quantity_per_product * oi.quantity * pr.unit_price, 2) AS component_total
FROM customer_orders AS o
JOIN order_items AS oi
    ON oi.order_id = o.order_id
JOIN products AS p
    ON p.product_id = oi.product_id
JOIN product_specification AS ps
    ON ps.product_id = oi.product_id
JOIN resources AS r
    ON r.resource_id = ps.resource_id
JOIN prices AS pr
    ON pr.resource_id = r.resource_id
    AND pr.product_id IS NULL
WHERE o.order_id = :order_id
ORDER BY oi.order_item_id, r.resource_type, r.name;
```

Приложение F. Приложение заданий 4-5

Файл: solutions/task4_auth_api/app.py

```
#!/usr/bin/env python3
"""Демо-модуль задания 4 и API задания 5 (только стандартная библиотека Python).

Запуск: python3 app.py -> http://127.0.0.1:8000
Демо-учётные записи: admin / Admin123! и user / User123!
Пароли хранятся только в виде PBKDF2-хеши. Замените демо-пароли для реального использования.
"""
from __future__ import annotations

import hashlib
import hmac
import html
import json
import os
import re
import secrets
import sqlite3
import threading
import time
from datetime import date
from http import HTTPStatus
from http.cookies import SimpleCookie
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from typing import Any
from urllib.parse import parse_qs, urlencode, urlsplit

APP_DIR = Path(__file__).resolve().parent
STATIC_DIR = APP_DIR / "static"
DB_PATH = Path(os.environ.get("EXAM_DB_PATH", APP_DIR / "exam_app.sqlite3")).resolve()
APP_PORT = int(os.environ.get("APP_PORT", "8000"))
PBKDF2_ROUNDS = 240_000
SESSION_TTL_SECONDS = 8 * 60 * 60
MAX_FORM_BYTES = 32_768
CAPTCHA_SIZE = 9

# Только для отдельного тестового процесса задания 5. В обычном режиме выключено.
FORCE_DB_ERROR = (
    os.environ.get("EXAM_TEST_MODE") == "1"
    and os.environ.get("EXAM_FORCE_DB_ERROR") == "1"
)

_sessions: dict[str, dict[str, Any]] = {}
_captcha_challenges: dict[str, float] = {}
_state_lock = threading.Lock()

SCHEMA = """
PRAGMA foreign_keys = ON;
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    login TEXT NOT NULL COLLATE NOCASE UNIQUE,
    password_salt TEXT NOT NULL,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL CHECK (role IN ('Администратор', 'Пользователь')),
    is_blocked INTEGER NOT NULL DEFAULT 0 CHECK (is_blocked IN (0, 1)),
    failed_attempts INTEGER NOT NULL DEFAULT 0 CHECK (failed_attempts >= 0),
    created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);
CREATE TABLE IF NOT EXISTS notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_notes_user_date ON notes(user_id, created_at DESC);
"""

def db_connect() -> sqlite3.Connection:
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)
    connection = sqlite3.connect(str(DB_PATH), timeout=5)
    connection.row_factory = sqlite3.Row
    connection.execute("PRAGMA foreign_keys = ON")
    connection.execute("PRAGMA busy_timeout = 5000")
    return connection

def make_password_hash(password: str, salt_hex: str | None = None) -> tuple[str, str]:
    salt_hex = salt_hex or secrets.token_hex(16)
    digest = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), bytes.fromhex(salt_hex), PBKDF2_ROUNDS
    ).hex()
    return salt_hex, digest

def password_matches(password: str, salt_hex: str, expected_hash: str) -> bool:
    _, actual = make_password_hash(password, salt_hex)
    return hmac.compare_digest(actual, expected_hash)

def initialize_database() -> None:
    """Создаёт схему и только при первом запуске добавляет тестовые записи."""
    connection = db_connect()
    try:
        connection.executescript(SCHEMA)
        with connection:
            count = connection.execute("SELECT COUNT(*) FROM users").fetchone()[0]
            if count == 0:
                for login, password, role in (
                    ("admin", os.environ.get("EXAM_ADMIN_PASSWORD", "Admin123!"), "Администратор"),
                    ("user", os.environ.get("EXAM_USER_PASSWORD", "User123!"), "Пользователь"),
                ):
                    salt, digest = make_password_hash(password)
                    connection.execute(
                        """INSERT INTO users(login, password_salt, password_hash, role)"""
                    )
    except Exception:
        pass
```

```

        VALUES (?, ?, ?, ?)""",
        (login, salt, digest, role),
    )
    if connection.execute("SELECT COUNT(*) FROM notes").fetchone()[0] == 0:
        users = {row["login"]: row["id"] for row in connection.execute(
            "SELECT id, login FROM users WHERE login IN ('admin', 'user')"}
        demo_notes = [
            ("Планирование", "Согласовать сроки подготовки проекта.", "admin", "2026-10-01"),
            ("Встреча", "Подготовить повестку и материалы к встрече.", "user", "2026-10-02"),
            ("Заметки по API", "Проверить формат JSON и коды ответов.", "admin", "2026-10-03"),
            ("Проверка базы", "Убедиться, что внешние ключи включены.", "user", "2026-10-04"),
            ("Итоги", "Сохранить результаты тестирования.", "admin", "2026-10-05"),
        ]
        for title, content, login, created_at in demo_notes:
            if login in users:
                connection.execute(
                    "INSERT INTO notes(title, content, user_id, created_at) VALUES (?, ?, ?, ?)",
                    (title, content, users[login], created_at),
                )
    finally:
        connection.close()

def _fetch_user(login: str) -> sqlite3.Row | None:
    connection = db_connect()
    try:
        return connection.execute(
            "SELECT * FROM users WHERE login = ? COLLATE NOCASE", (login,)
        ).fetchone()
    finally:
        connection.close()

def _record_failed_attempt(login: str) -> bool:
    """Увеличивает серию неудач; возвращает True, если запись заблокирована."""
    if not login:
        return False
    connection = db_connect()
    try:
        with connection:
            connection.execute(
                """UPDATE users
                SET failed_attempts = failed_attempts + 1,
                is_blocked = CASE WHEN failed_attempts + 1 >= 3 THEN 1 ELSE 0 END
                WHERE login = ? COLLATE NOCASE AND is_blocked = 0""",
                (login,)
            )
        row = connection.execute(
            "SELECT is_blocked FROM users WHERE login = ? COLLATE NOCASE", (login,)
        ).fetchone()
        return bool(row and row["is_blocked"])
    finally:
        connection.close()

def _reset_failed_attempts(user_id: int) -> None:
    connection = db_connect()
    try:
        with connection:
            connection.execute(
                "UPDATE users SET failed_attempts = 0 WHERE id = ?", (user_id,)
            )
    finally:
        connection.close()

def _read_form(handler: BaseHTTPRequestHandler) -> dict[str, str]:
    try:
        length = int(handler.headers.get("Content-Length", "0"))
    except ValueError:
        return {}
    if length < 0 or length > MAX_FORM_BYTES:
        return {}
    raw = handler.rfile.read(length).decode("utf-8", errors="replace")
    return {key: values[-1] for key, values in parse_qs(raw, keep_blank_values=True).items()}

def _parse_captcha_order(value: str) -> list[int] | None:
    try:
        parsed = json.loads(value)
        if not isinstance(parsed, list) or len(parsed) != CAPTCHA_SIZE:
            return None
        if any(isinstance(item, str) for item in parsed):
            return None
        if sorted(parsed) != list(range(CAPTCHA_SIZE)):
            return None
        return parsed
    except (json.JSONDecodeError, TypeError):
        return None

def _clean_expired_state() -> None:
    now = time.time()
    with _state_lock:
        for token in [key for key, value in _captcha_challenges.items() if value < now]:
            _captcha_challenges.pop(token, None)
        for token in [key for key, value in _sessions.items() if value["expires"] < now]:
            _sessions.pop(token, None)

class ExamRequestHandler(BaseHTTPRequestHandler):
    server_version = "ExamDemo/1.0"

    def log_message(self, fmt: str, *args: Any) -> None:
        # Не выводим введенные пользователем пароли в журнал.
        super().log_message(fmt, *args)

    def _common_headers(self) -> None:
        self.send_header("X-Content-Type-Options", "nosniff")
        self.send_header("X-Frame-Options", "DENY")

```

```

self.send_header("Referrer-Policy", "no-referrer")
self.send_header("Cache-Control", "no-store")

def _send_html(self, body: str, status: int = 200, extra_headers: list[tuple[str, str]] | None = None) -> None:
    encoded = body.encode("utf-8")
    self.send_response(status)
    self.send_header("Content-Type", "text/html; charset=utf-8")
    self.send_header("Content-Length", str(len(encoded)))
    self._common_headers()
    for key, value in extra_headers or []:
        self.send_header(key, value)
    self.end_headers()
    self.wfile.write(encoded)

def _send_json(self, payload: Any, status: int = 200, extra_headers: list[tuple[str, str]] | None = None) -> None:
    encoded = json.dumps(payload, ensure_ascii=False, separators=(",", ":")).encode("utf-8")
    self.send_response(status)
    self.send_header("Content-Type", "application/json; charset=utf-8")
    self.send_header("Content-Length", str(len(encoded)))
    self._common_headers()
    for key, value in extra_headers or []:
        self.send_header(key, value)
    self.end_headers()
    self.wfile.write(encoded)

def _redirect(self, location: str, cookie: str | None = None) -> None:
    self.send_response(HTTPStatus.SEE_OTHER)
    self.send_header("Location", location)
    self.send_header("Content-Length", "0")
    self._common_headers()
    if cookie:
        self.send_header("Set-Cookie", cookie)
    self.end_headers()

def _current_user(self) -> sqlite3.Row | None:
    _clean_expired_state()
    _cookie = SimpleCookie()
    try:
        cookie.load(self.headers.get("Cookie", ""))
    except Exception:
        return None
    morsel = cookie.get("sid")
    if not morsel:
        return None
    with _state_lock:
        session = _sessions.get(morsel.value)
    if not session or session["expires"] < time.time():
        return None
    return _fetch_user_by_id(session["user_id"])

def do_GET(self) -> None: # noqa: N802 - имя задано BaseHTTPRequestHandler
    parsed = urlsplit(self.path)
    path = parsed.path
    if path in ("/notes", "/api/notes"):
        self._get_notes(parsed.query)
        return
    if path == "/static/captcha_full.png":
        asset = STATIC_DIR / "captcha_full.png"
        try:
            data = asset.read_bytes()
        except OSError:
            self._send_html(_page("Ошибка", "Файл изображения капчи не найден."), 500)
            return
        self.send_response(200)
        self.send_header("Content-Type", "image/png")
        self.send_header("Content-Length", str(len(data)))
        self._common_headers()
        self.end_headers()
        self.wfile.write(data)
        return
    if path in ("/", "/login"):
        user = self._current_user()
        if user:
            self._redirect("/dashboard")
        else:
            self._login_page()
        return
    if path == "/dashboard":
        user = self._current_user()
        if not user:
            self._redirect("/login")
        else:
            welcome = parse_qs(parsed.query).get("welcome", [""])[0] == "1"
            self._dashboard(user, welcome=welcome)
        return
    if path == "/admin/users":
        user = self._current_user()
        if not user:
            self._redirect("/login")
        elif user["role"] != "Администратор":
            self._send_html(_page("Доступ запрещён", "Эта страница доступна только администратору."), 403)
        else:
            self._admin_page(user)
        return
    self._send_html(_page("Не найдено", "Страница не найдена."), 404)

def do_POST(self) -> None: # noqa: N802
    path = urlsplit(self.path).path
    if path in ("/notes", "/api/notes"):
        self._send_json({"error": "Метод не поддерживается. Используйте GET."}, 405,
            [{"Allow", "GET"}])
        return
    if path == "/login":
        self._login(_read_form(self))
        return
    if path == "/logout":
        self._logout()
        return
    if path == "/admin/users/create":
        self._admin_create_user(_read_form(self))
        return

```

```

if path == "/admin/users/update":
    self._admin_update_user(_read_form(self))
    return
self._send_html(_page("Не найдено", "Страница не найдена."), 404)

def _get_notes(self, raw_query: str) -> None:
    """GET /notes. Все ошибки данного маршрута всегда возвращаются как JSON."""
    query = parse_qs(raw_query, keep_blank_values=True)
    allowed = {"limit", "user_id"}
    unknown = sorted(set(query) - allowed)
    if unknown:
        self._send_json({"error": "Неизвестный параметр: " + ", ".join(unknown)}, 400)
        return
    if any(len(values) != 1 for values in query.values()):
        self._send_json({"error": "Каждый параметр можно передать только один раз."}, 400)
        return

    limit: int | None = None
    user_id: int | None = None
    try:
        if "limit" in query:
            limit = int(query["limit"][0])
            if not 1 <= limit <= 1000:
                raise ValueError
        if "user_id" in query:
            user_id = int(query["user_id"][0])
            if user_id <= 0:
                raise ValueError
    except ValueError:
        self._send_json(
            {"error": "Параметры limit и user_id должны быть положительными целыми числами; limit — от 1 до 1000."},
            400,
        )
        return

    connection: sqlite3.Connection | None = None
    try:
        if FORCE_DB_ERROR:
            raise sqlite3.OperationalError("Тестовая имитация сбоя подключения к БД")
        connection = db_connect()
        sql = """SELECT n.id, n.title, n.content, n.created_at, u.login
            FROM notes AS n JOIN users AS u ON u.id = n.user_id"""
        conditions: list[str] = []
        parameters: list[Any] = []
        if user_id is not None:
            conditions.append("n.user_id = ?")
            parameters.append(user_id)
        if conditions:
            sql += " WHERE " + " AND ".join(conditions)
        sql += " ORDER BY n.created_at DESC, n.id DESC"
        if limit is not None:
            sql += " LIMIT ?"
            parameters.append(limit)
        rows = connection.execute(sql, parameters).fetchall()
        result = []
        for row in rows:
            created = date.fromisoformat(str(row["created_at"])[0:10])
            result.append({
                "id": int(row["id"]),
                "title_user": f'{row["title"]} - {row["login"]}',
                "content": row["content"],
                "formatted_date": created.strftime("%d.%m.%Y"),
            })
        self._send_json(result, 200)
    except Exception as error:
        # Деталь исключения остаётся в серверном журнале, клиент получает безопасное JSON-сообщение.
        print(f"Ошибка GET /notes: {type(error).__name__}: {error}", flush=True)
        self._send_json({"error": "Ошибка подключения к базе данных"}, 500)
    finally:
        if connection is not None:
            connection.close()

def _login_page(self, message: str = "", login_value: str = "", success: bool = False) -> None:
    _clean_expired_state()
    challenge_id = secrets.token_urlsafe(18)
    initial_order = list(range(CAPTCHA_SIZE))
    # Случайно перемешиваем стартовое расположение; решение проверяется сервером.
    import random
    random.SystemRandom().shuffle(initial_order)
    with _state_lock:
        captcha_challenges[challenge_id] = time.time() + 5 * 60
    tiles = []
    for piece in initial_order:
        x, y = piece % 3, piece // 3
        tiles.append(
            f'<button type="button" class="puzzle-piece" draggable="true" '
            f'data-piece="{piece}" aria-label="Фрагмент {piece + 1}" '
            f'style="background-position:{x * 50}% {y * 50}%"></button>'
        )
    notice_class = "notice success" if success else "notice error"
    if success:
        notice_title, notice_icon = "Информация", "✓"
    elif "заблокированы" in message.lower():
        notice_title, notice_icon = "Блокировка", "⏹"
    elif "капча" in message.lower():
        notice_title, notice_icon = "Проверка", "△"
    else:
        notice_title, notice_icon = "Ошибка", "△"
    notice = (f'<div class="{notice_class}" role="status"><strong>{notice_icon} '
            f'{notice_title}</strong><br>{html.escape(message)}</div>') if message else ""
    safe_login = html.escape(login_value, quote=True)
    initial_json = html.escape(json.dumps(initial_order), quote=True)
    body = f'<{doctype html}>
<html lang="ru"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Авторизация</title><style>
:root{{--ink:#172033;--blue:#4058d6;--muted:#67718a;--line:#dcdce2;--paper:#fff;--bg:#f3f6fb}}
*{{box-sizing:border-box}}body{{margin:0;background:linear-gradient(140deg,#edf2ff,#f8fbff 52%,#eef8f6);font:16px/1.45
system-ui,-apple-system,"Segoe UI",sans-serif;color:var(--ink);min-height:100vh;display:grid;place-items:center;padding:
22px}}
.card{{width:min(100%,510px);background:var(--paper);border:1px solid #e5e9f2;border-radius:22px;box-shadow:0 20px 60px
#25366618;padding:30px}}

```

```

.kicker{{color:var(--blue);font-size:12px;font-weight:800;letter-spacing:.12em;text-transform:uppercase}}
h1{{font-size:28px;margin:7px 0 4px}}.muted{{color:var(--muted);margin:0 0 22px}}label{{display:block;font-weight:650;
margin:14px 0 6px}}
input[type=text],input[type=password]{{width:100%;border:1px solid var(--line);border-radius:11px;padding:12px 13px;
font:inherit;outline:none}}
input:focus{{border-color:var(--blue);box-shadow:0 0 3px #4058d61c}}.notice{{padding:11px 13px;border-radius:10px;
margin:12px 0}}.error{{background:#fff0ee;color:#a1251b}}.success{{background:#e9f8ee;color:#166534}}
.puzzle-wrap{{margin:10px 0 15px;padding:14px;background:#f5f7fc;border:1px solid var(--line);border-radius:14px}}
.puzzle-instruction{{margin:0 0 11px;font-size:13px;color:var(--muted)}}.puzzle{{display:grid;grid-template-columns:
repeat(3,76px);gap:4px;justify-content:center;touch-action:manipulation}}
.puzzle-piece{{width:76px;height:76px;border:1px solid #8091b7;border-radius:6px;background-color:#edf4ff;background-
image:url('/static/captcha_full.png');background-size:300% 300%;cursor:grab;padding:0;transition:transform .12s;border-
color .12s}}
.puzzle-piece:hover{{border:2px solid var(--blue)}}.puzzle-piece.selected{{outline:3px solid #4058d6;outline-offset:1px;
transform:scale(.96)}}
button.submit{{width:100%;margin-top:13px;border:0;border-radius:11px;padding:13px;background:var(--blue);color:white;
font:inherit;font-weight:750;cursor:pointer}}
button.submit:hover{{background:#3248c3}}.footer{{margin-top:18px;padding-top:14px;border-top:1px solid var(--line);
font-size:12px;color:var(--muted)}}
@media(max-width:420px){{.card{{padding:22px}}.puzzle{{grid-template-columns:repeat(3,64px)}}.puzzle-piece{{width:64px;
height:64px}}}}
</style></head><body><main class="card"><div class="kicker">Система управления</div><h1>Вход в систему</h1>
<p class="muted">Введите учётные данные и соберите изображение из фрагментов.</p><notice>
<form method="post" action="/login" autocomplete="on">
<label for="login">Логин</label><input id="login" name="login" type="text" minlength="1" maxlength="32" required
autocomplete="username" value="{safe_login}">
<label for="password">Пароль</label><input id="password" name="password" type="password" required autocomplete="current-
password">
<div class="puzzle-wrap"><p class="puzzle-instruction">Нажмите на два фрагмента, чтобы поменять их местами, или
перетяните фрагмент. Соберите исходную картинку.</p>
<div class="puzzle" id="puzzle" aria-label="Интерактивная капча">{{.join(tiles)}}</div></div>
<input type="hidden" name="captcha_id" value="{challenge_id}">
<input type="hidden" id="captcha_order" name="captcha_order" value="{initial_json}">
<button class="submit" type="submit">Войти</button></form>
<div class="footer">После трёх последовательных ошибок учётная запись блокируется. Обратитесь к администратору для
снятия блокировки.</div>
</main><script>
({})=>{{const board=document.getElementById('puzzle'),out=document.getElementById('captcha_order');let selected=null,
dragged=null;
function save(){{out.value=JSON.stringify([...board.children].map(tile=>Number(tile.dataset.piece)))}}
function swap(a,b){{if(!a||!b||a===b)return;const marker=document.createElement('span');board.insertBefore(marker,a);
board.insertBefore(b,marker);board.insertBefore(b,marker);marker.remove();a.classList.remove('selected');b.classList.remove(
'selected');selected=null;save()}}
board.querySelectorAll('.puzzle-piece').forEach(tile=>{{tile.addEventListener('click',()=>{{if(!selected){{
selected=tile;tile.classList.add('selected')}}else if(selected===tile){{tile.classList.remove('selected')}}
selected=null}}else swap(selected,tile)}});
tile.addEventListener('dragstart',e=>{{(dragged=tile;e.dataTransfer.effectAllowed='move';e.dataTransfer.setData('text/
plain',tile.dataset.piece)}});
tile.addEventListener('dragover',e=>e.preventDefault());tile.addEventListener('drop',e=>{{e.preventDefault();const
source=[...board.children].find(x=>x.dataset.piece===e.dataTransfer.getData('text/plain'));swap(source,tile)}})}};save(
)}}})();
</script></body></html>""
self._send_html(body)

def _login(self, form: dict[str, str]) -> None:
    login = form.get("login", "").strip().casefold()
    password = form.get("password", "")
    user = _fetch_user(login) if login else None
    if user and user["is_blocked"]:
        self._login_page("Вы заблокированы. Обратитесь к администратору", login)
        return

    challenge_id = form.get("captcha_id", "")
    with _state_lock:
        expires = _captcha_challenges.pop(challenge_id, None)
        captcha_order = _parse_captcha_order(form.get("captcha_order", ""))
        captcha_valid = expires is not None and expires >= time.time() and captcha_order == list(range(CAPTCHA_SIZE))
    if not captcha_valid:
        locked = _record_failed_attempt(login)
        message = (
            "Вы заблокированы. Обратитесь к администратору"
            if locked else "Капча собрана неверно или время её действия истекло. Переставьте фрагменты и попробуйте
снова."
        )
        self._login_page(message, login)
        return

    correct_password = bool(
        user and password_matches(password, user["password_salt"], user["password_hash"])
    )
    if not user or not correct_password:
        locked = _record_failed_attempt(login)
        message = (
            "Вы заблокированы. Обратитесь к администратору"
            if locked else "Вы ввели неверный логин или пароль. Пожалуйста проверьте ещё раз введенные данные"
        )
        self._login_page(message, login)
        return

    _reset_failed_attempts(int(user["id"]))
    session_id = secrets.token_urlsafe(32)
    with _state_lock:
        _sessions[session_id] = {
            "user_id": int(user["id"]),
            "expires": time.time() + SESSION_TTL_SECONDS,
        }
    cookie = f"sid={session_id}; Path=/; HttpOnly; SameSite=Lax; Max-Age={SESSION_TTL_SECONDS}"
    # Успешная авторизация подтверждается точным текстом из задания.
    self._redirect("/dashboard?welcome=1", cookie)

def _dashboard(self, user: sqlite3.Row, welcome: bool = False) -> None:
    login = html.escape(user["login"])
    role = html.escape(user["role"])
    banner = '<div class="notice success">Вы успешно авторизовались</div>' if welcome else ""
    admin_link = '<p><a class="button secondary" href="/admin/users">Управление пользователями</a></p>' if user["role"] == "Администратор" else ""
    body = f""<main class="card"><div class="kicker">Рабочий стол</div><h1>Здравствуй, {login}</h1>{banner}
<p>Роль: <strong>{role}</strong></p><p>API заметок: <a href="/notes">GET /notes</a></p>{admin_link}
<form method="post" action="/logout"><button class="button" type="submit">Выйти</button></form></main>""
    self._send_html(_page("Рабочий стол", body, styled=True))

def _logout(self) -> None:

```

```

cookie = SimpleCookie()
try:
    cookie.load(self.headers.get("Cookie", ""))
except Exception:
    cookie = SimpleCookie()
morsel = cookie.get("sid")
if morsel:
    with _state_lock:
        sessions.pop(morsel.value, None)
self._redirect("/login", "sid=" + Path=/; HttpOnly; SameSite=Lax; Max-Age=0")

def _admin_page(self, current_user: sqlite3.Row, message: str = "", is_error: bool = False) -> None:
    connection = db_connect()
    try:
        users = connection.execute(
            "SELECT id, login, role, is_blocked, failed_attempts FROM users ORDER BY id"
        ).fetchall()
    finally:
        connection.close()
    banner = ""
    if message:
        cls = "notice error" if is_error else "notice success"
        title = "⚠ Ошибка" if is_error else "✓ Информация"
        banner = f'<div class="{cls}" role="status"><strong>{title}</strong><br>{html.escape(message)}</div>'
    rows = []
    for user in users:
        role_options = "".join(
            f'<option value="{role}"{" selected" if user["role"] == role else ""}>{role}</option>'
            for role in ("Пользователь", "Администратор")
        )
        blocked_options = (
            f'<option value="0"{" selected" if not user["is_blocked"] else ""}>Активна</option>'
            f'<option value="1"{" selected" if user["is_blocked"] else ""}>Заблокирована</option>'
        )
        rows.append(f'<tr><td>{int(user["id"])}</td><td>{html.escape(user["login"])}</td>
<td>{int(user["failed_attempts"])}</td><td><form class="edit" method="post" action="/admin/users/update">
<input type="hidden" name="user_id" value="{int(user["id"])}"><input name="login" value="{html.escape(user["login"])"
quote=True}</input> required minlength="1" maxlength="32" pattern="[A-Za-zA-Яa-яЁё0-9_.-]+" aria-label="Логин">
<select name="role">{role_options}</select><select name="is_blocked">{blocked_options}</select>
<input name="new_password" type="password" minlength="8" placeholder="Новый пароль (необязательно)">
<button class="button small" type="submit">Сохранить</button></form></td></tr>'")
    table_rows = "".join(rows)
    body = f'<main class="wrap"><a href="/dashboard"> Рабочий стол</a><div class="kicker">Администрирование</div>
<h1>Пользователи</h1><div>{banner}</div><section class="panel"><h2>Добавить пользователя</h2>
<form class="create" method="post" action="/admin/users/create"><label>Логин<input name="login" required minlength="1"
maxlength="32" pattern="[A-Za-zA-Яa-яЁё0-9_.-]+"></label>
<label>Пароль<input name="password" type="password" required minlength="8"></label><label>Роль<select
name="role"><option>Пользователь</option><option>Администратор</option></select></label>
<button class="button" type="submit">Добавить</button></form></section>
<section class="panel"><h2>Текущие учётные записи</h2><div class="table-scroll"><table><thead><tr><th>ID</th><th>Логин</th>
<th>Ошибки</th><th>Роль</th><th>Поль / блокировка</th><th>Пароль</th></tr><tbody><tr><td>{table_rows}</tbody></table></div></section>
</main>'")
    self._send_html(_page("Управление пользователями", body, styled=True, admin=True))

def _authorized_admin(self) -> sqlite3.Row | None:
    user = self._current_user()
    if not user or user["role"] != "Администратор":
        return None
    return user

def _admin_create_user(self, form: dict[str, str]) -> None:
    admin = self._authorized_admin()
    if not admin:
        self._send_html(_page("Доступ запрещён", "Войдите под учётной записью администратора.", 403))
        return
    login = form.get("login", "").strip().casefold()
    password = form.get("password", "")
    role = form.get("role", "")
    if not re.fullmatch(r"[A-Za-zA-Яa-яЁё0-9_.-]{1,32}", login):
        self._admin_page(admin, "Логин: 1-32 буквы, цифры, точка, дефис или подчёркивание.", True)
        return
    if len(password) < 8:
        self._admin_page(admin, "Пароль должен содержать не менее 8 символов.", True)
        return
    if role not in ("Пользователь", "Администратор"):
        self._admin_page(admin, "Выберите допустимую роль.", True)
        return
    salt, digest = make_password_hash(password)
    connection = db_connect()
    try:
        with connection:
            connection.execute(
                "INSERT INTO users(login, password_salt, password_hash, role) VALUES (?, ?, ?, ?)",
                (login, salt, digest, role),
            )
        self._admin_page(admin, f"Пользователь «{login}» добавлен.")
    except sqlite3.IntegrityError:
        self._admin_page(admin, "Пользователь с указанным логином уже существует. Выберите другой логин.", True)
    finally:
        connection.close()

def _admin_update_user(self, form: dict[str, str]) -> None:
    admin = self._authorized_admin()
    if not admin:
        self._send_html(_page("Доступ запрещён", "Войдите под учётной записью администратора.", 403))
        return
    try:
        user_id = int(form.get("user_id", ""))
        blocked = int(form.get("is_blocked", ""))
    except ValueError:
        self._admin_page(admin, "Некорректный идентификатор или статус блокировки.", True)
        return
    login = form.get("login", "").strip().casefold()
    role = form.get("role", "")
    new_password = form.get("new_password", "")
    if not re.fullmatch(r"[A-Za-zA-Яa-яЁё0-9_.-]{1,32}", login):
        self._admin_page(admin, "Логин: 1-32 буквы, цифры, точка, дефис или подчёркивание.", True)
        return
    if user_id <= 0 or blocked not in (0, 1) or role not in ("Пользователь", "Администратор"):
        self._admin_page(admin, "Проверьте ID, логин, роль и статус блокировки.", True)
        return
    if new_password and len(new_password) < 8:

```



```

        self._admin_page(admin, "Новый пароль должен содержать не менее 8 символов.", True)
    return
if user_id == int(admin["id"]) and (blocked or role != "Администратор"):
    self._admin_page(admin, "Нельзя заблокировать или лишить роли текущую учётную запись администратора.", True)
    return

connection = db_connect()
try:
    target = connection.execute("SELECT id, login FROM users WHERE id = ?", (user_id,)).fetchone()
    if not target:
        self._admin_page(admin, "Пользователь не найден. Обновите страницу и повторите действие.", True)
        return
    with connection:
        if new_password:
            salt, digest = make_password_hash(new_password)
            connection.execute(
                """UPDATE users SET login = ?, role = ?, is_blocked = ?, failed_attempts = 0,
                password_salt = ?, password_hash = ? WHERE id = ?""",
                (login, role, blocked, salt, digest, user_id),
            )
        else:
            connection.execute(
                """UPDATE users SET login = ?, role = ?, is_blocked = ?,
                failed_attempts = CASE WHEN ? = 0 THEN 0 ELSE failed_attempts END
                WHERE id = ?""",
                (login, role, blocked, blocked, user_id),
            )
        self._admin_page(admin, f"Данные пользователя «{login}» обновлены.")
except sqlite3.IntegrityError:
    self._admin_page(admin, "Пользователь с указанным логином уже существует. Укажите уникальный логин.", True)
finally:
    connection.close()

def _fetch_user_by_id(user_id: int) -> sqlite3.Row | None:
    connection = db_connect()
    try:
        return connection.execute("SELECT * FROM users WHERE id = ?", (user_id,)).fetchone()
    finally:
        connection.close()

def _page(title: str, body: str, styled: bool = False, admin: bool = False) -> str:
    if not styled:
        return f"""<!doctype html><html lang="ru"><head><meta charset="utf-8"><meta name="viewport"
content="width=device-width,initial-scale=1"><title>{html.escape(title)}</title></head><body><h1>{html.escape(title)}</
h1></body></html>"""
    extra = """
<style>
:root{--ink:#172033;--blue:#4058d6;--muted:#67718a;--line:#dce2ee;--paper:#fff;--bg:#f3f6fb}
*{box-sizing:border-box}body{margin:0;background:var(--bg);font:15px/1.5 system-ui,-apple-system,"Segoe UI",sans-serif;
color:var(--ink);padding:28px}
.card,.wrap{max-width:900px;margin:5vh auto;background:white;border:1px solid #e5e9f2;border-radius:20px;padding:28px;
box-shadow:0 16px 50px #25366614}
.wrap{max-width:1150px;margin:0 auto}.kicker{color:var(--blue);font-size:12px;font-weight:800;letter-spacing:.12em;text-
transform:uppercase}h1{margin:7px 0 20px;font-size:28px}h2{font-size:18px;margin:0 0 12px}
a{color:var(--blue)}.button{display:inline-block;border:0;border-radius:9px;padding:10px 15px;background:var(--blue);
color:#fff;text-decoration:none;font:inherit;font-weight:700;cursor:pointer}
.button.secondary{background:#e9edff;color:#3046bd}.button.small{padding:8px 11px;font-size:13px}.notice{padding:11px
13px;border-radius:10px;margin:12px 0}.success{background:#e9f8ee;color:#166534}.error{background:#fff0ee;color:#a1251b}
.panel{background:#fff;border:1px solid var(--line);border-radius:14px;padding:18px;margin:16px 0}.create{display:flex;
gap:12px;align-items:end;flex-wrap:wrap}.create label{display:grid;gap:5px;font-weight:650}.create input,.create select,
.edit input,.edit select{border:1px solid var(--line);border-radius:8px;padding:8px 10px;font:inherit}
.table-scroll{overflow:auto}table{width:100%;border-collapse:collapse;min-width:760px}th,td{text-align:left;padding:
10px;border-bottom:1px solid var(--line);vertical-align:middle}th{font-size:13px;color:var(--muted)}.edit{display:flex;
gap:8px;align-items:center;flex-wrap:wrap}
</style>""" if admin else ""
</style>
:root{--ink:#172033;--blue:#4058d6;--bg:#f3f6fb}*{box-sizing:border-box}body{margin:0;background:var(--bg);font:16px/1.
5 system-ui,-apple-system,"Segoe UI",sans-serif;color:var(--ink);padding:22px}
.card{max-width:620px;margin:9vh auto;background:white;border:1px solid #e5e9f2;border-radius:20px;padding:30px;box-
shadow:0 16px 50px #25366614}.kicker{color:var(--blue);font-size:12px;font-weight:800;letter-spacing:.12em;text-
transform:uppercase}h1{margin:7px 0 20px;font-size:28px}
a{color:var(--blue)}.button{display:inline-block;border:0;border-radius:9px;padding:10px 15px;background:var(--blue);
color:#fff;text-decoration:none;font:inherit;font-weight:700;cursor:pointer}.button.secondary{background:#e9edff;color:
#3046bd}.notice{padding:11px 13px;border-radius:10px;margin:12px 0}.success{background:#e9f8ee;color:#166534}
</style>"""
    return f"""<!doctype html><html lang="ru"><head><meta charset="utf-8"><meta name="viewport" content="width=device-
width,initial-scale=1"><title>{html.escape(title)}</title>{extra}</head><body>{body}</body></html>"""

def make_server(host: str = "127.0.0.1", port: int = APP_PORT) -> ThreadingHTTPServer:
    return ThreadingHTTPServer((host, port), ExamRequestHandler)

def main() -> None:
    initialize_database()
    server = make_server("0.0.0.0", APP_PORT)
    print(f"Сервер запущен: http://127.0.0.1:{APP_PORT}")
    print("Демо-входы: admin / Admin123! и user / User123! (поменяйте перед реальным использованием)")
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nОстановка сервера...")
    finally:
        server.server_close()

if __name__ == "__main__":
    main()

```

Приложение G. Автоматические тесты

Файл: solutions/task4_auth_api/tests/test_app.py

```
import json
import re
import sys
import tempfile
import threading
import unittest
from http.cookiejar import CookieJar
from urllib.request import HTTPCookieProcessor, build_opener
from pathlib import Path
from urllib.error import HTTPError
from urllib.parse import urlencode
from urllib.request import Request, urlopen

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import app # noqa: E402

class ExamApplicationTests(unittest.TestCase):
    def setUp(self):
        self.temp = tempfile.TemporaryDirectory()
        self.old_db_path = app.DB_PATH
        self.old_force_error = app.FORCE_DB_ERROR
        app.DB_PATH = Path(self.temp.name) / "test.sqlite3"
        app.FORCE_DB_ERROR = False
        app.sessions.clear()
        app.captcha_challenges.clear()
        app.initialize_database()
        self.server = app.make_server("127.0.0.1", 0)
        self.thread = threading.Thread(target=self.server.serve_forever, daemon=True)
        self.thread.start()
        self.base = f"http://127.0.0.1:{self.server.server_port}"

    def tearDown(self):
        self.server.shutdown()
        self.server.server_close()
        self.thread.join(timeout=2)
        app.DB_PATH = self.old_db_path
        app.FORCE_DB_ERROR = self.old_force_error
        self.temp.cleanup()

    def fetch(self, path):
        with urlopen(self.base + path, timeout=5) as response:
            return response.status, response.headers, response.read()

    def test_notes_returns_expected_json_and_date(self):
        status, headers, body = self.fetch("/notes")
        payload = json.loads(body.decode("utf-8"))
        self.assertEqual(status, 200)
        self.assertIn("application/json", headers.get("Content-Type", ""))
        self.assertGreaterEqual(len(payload), 3)
        self.assertEqual(set(payload[0]), {"id", "title_user", "content", "formatted_date"})
        self.assertRegex(payload[0]["formatted_date"], r"^\d{2}\.\d{2}\.\d{4}$")
        self.assertIn(" - ", payload[0]["title_user"])

    def test_empty_notes_is_successful_empty_array(self):
        connection = app.db_connect()
        try:
            with connection:
                connection.execute("DELETE FROM notes")
        finally:
            connection.close()
        status, headers, body = self.fetch("/notes")
        self.assertEqual(status, 200)
        self.assertEqual(json.loads(body.decode("utf-8")), [])
        self.assertIn("application/json", headers.get("Content-Type", ""))

    def test_invalid_limit_returns_400_json(self):
        with self.assertRaises(HTTPError) as caught:
            urlopen(self.base + "/notes?limit=abc", timeout=5)
        self.assertEqual(caught.exception.code, 400)
        self.assertIn("application/json", caught.exception.headers.get("Content-Type", ""))
        self.assertIn("error", json.loads(caught.exception.read().decode("utf-8")))

    def test_database_failure_returns_500_json(self):
        app.FORCE_DB_ERROR = True
        with self.assertRaises(HTTPError) as caught:
            urlopen(self.base + "/notes", timeout=5)
        self.assertEqual(caught.exception.code, 500)
        payload = json.loads(caught.exception.read().decode("utf-8"))
        self.assertEqual(payload["error"], "Ошибка подключения к базе данных")
        self.assertIn("application/json", caught.exception.headers.get("Content-Type", ""))

    def test_three_consecutive_bad_passwords_block_user(self):
        last_body = ""
        for _ in range(3):
            with urlopen(self.base + "/login", timeout=5) as response:
                page = response.read().decode("utf-8")
                challenge = re.search(r'name="captcha_id" value="([^\"]+)"', page).group(1)
                form = urlencode({
                    "login": "user",
                    "password": "definitely-wrong",
                    "captcha_id": challenge,
                    "captcha_order": json.dumps(list(range(9))),
                }).encode("utf-8")
                request = Request(self.base + "/login", data=form,
                                   headers={"Content-Type": "application/x-www-form-urlencoded"})
            with urlopen(request, timeout=10) as response:
                last_body = response.read().decode("utf-8")
        self.assertIn("Вы заблокированы. Обратитесь к администратору", last_body)
        connection = app.db_connect()
        try:
            row = connection.execute("SELECT is_blocked, failed_attempts FROM users WHERE login='user'").fetchone()
            self.assertEqual((row["is_blocked"], row["failed_attempts"]), (1, 3))
        finally:
            connection.close()

    def test_non_get_api_method_returns_405(self):
        pass
```

```

request = Request(self.base + "/notes", data=b"", method="POST")
with self.assertRaises(HTTPErrors) as caught:
    urlopen(request, timeout=5)
self.assertEqual(caught.exception.code, 405)
self.assertEqual(caught.exception.headers.get("Allow"), "GET")
self.assertIn("application/json", caught.exception.headers.get("Content-Type", ""))

def test_admin_can_add_edit_and_unlock_users(self):
    opener = build_opener(HTTPCookieProcessor(CookieJar()))
    page = opener.open(self.base + "/login", timeout=5).read().decode("utf-8")
    challenge = re.search(r'name="captcha_id" value="([^\"]+)"', page).group(1)
    login_form = urlencode({
        "login": "admin",
        "password": "Admin123!",
        "captcha_id": challenge,
        "captcha_order": json.dumps(list(range(9))),
    }).encode("utf-8")
    request = Request(self.base + "/login", data=login_form,
        headers={"Content-Type": "application/x-www-form-urlencoded"})
    dashboard = opener.open(request, timeout=10).read().decode("utf-8")
    self.assertIn("Вы успешно авторизовались", dashboard)

    create_form = urlencode({"login": "alice", "password": "Alice123!", "role": "Пользователь"}).encode("utf-8")
    create_request = Request(self.base + "/admin/users/create", data=create_form,
        headers={"Content-Type": "application/x-www-form-urlencoded"})
    created = opener.open(create_request, timeout=10).read().decode("utf-8")
    self.assertIn("Пользователь «alice» добавлен", created)

    connection = app.db_connect()
    try:
        user_id = connection.execute("SELECT id FROM users WHERE login='user'").fetchone()["id"]
    finally:
        connection.close()
    edit_form = urlencode({"user_id": user_id, "login": "user-renamed", "role": "Пользователь",
        "is_blocked": 1, "new_password": ""}).encode("utf-8")
    edit_request = Request(self.base + "/admin/users/update", data=edit_form,
        headers={"Content-Type": "application/x-www-form-urlencoded"})
    edited = opener.open(edit_request, timeout=10).read().decode("utf-8")
    self.assertIn("Данные пользователя «user-renamed» обновлены", edited)
    connection = app.db_connect()
    try:
        row = connection.execute("SELECT is_blocked FROM users WHERE login='user-renamed'").fetchone()
        self.assertEqual(row["is_blocked"], 1)
    finally:
        connection.close()

    unlock_form = urlencode({"user_id": user_id, "login": "user-renamed", "role": "Пользователь",
        "is_blocked": 0, "new_password": ""}).encode("utf-8")
    unlock_request = Request(self.base + "/admin/users/update", data=unlock_form,
        headers={"Content-Type": "application/x-www-form-urlencoded"})
    opener.open(unlock_request, timeout=10).read()
    connection = app.db_connect()
    try:
        row = connection.execute("SELECT is_blocked, failed_attempts FROM users WHERE login='user-renamed'").
fetchone()
        self.assertEqual((row["is_blocked"], row["failed_attempts"]), (0, 0))
    finally:
        connection.close()

if __name__ == "__main__":
    unittest.main(verbosity=2)

```

Приложение Н. Коллекция Postman

Файл: solutions/task5_api/Exam_API.postman_collection.json

```
{
  "info": {
    "postman_id": "6e8dca90-0f1a-4f6c-8a5c-4e7da8baf307",
    "name": "Экзамен – API заметок",
    "description": "Проверяет успешный ответ GET /notes, валидацию параметра и JSON-ответ при внутренней ошибке. Для теста 500 запустите отдельный тестовый экземпляр, см. README задания 5.",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "GET /notes – 200 и корректный JSON",
      "request": {
        "method": "GET",
        "header": [],
        "url": "{{baseUrl}}/notes"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "type": "text/javascript",
            "exec": [
              "pm.test('Статус 200', function () { pm.response.to.have.status(200); });",
              "pm.test('Ответ имеет Content-Type JSON', function () { pm.expect(pm.response.headers.get('Content-Type')).to.include('application/json'); });",
              "pm.test('Тело – валидный JSON-массив', function () { pm.response.to.be.json; pm.expect(pm.response.json()).to.be.an('array'); });",
              "pm.test('Поля преобразованной заметки и дата ДД.ММ.ГГГГ', function () { const data = pm.response.json(); if (data.length) { pm.expect(data[0]).to.have.all.keys('id', 'title_user', 'content', 'formatted_date'); pm.expect(data[0].title_user).to.include(' - '); pm.expect(data[0].formatted_date).to.match(/^\d{2}\\.\d{2}\\.\d{4}$/); } });"
            ]
          }
        }
      ]
    },
    {
      "name": "GET /notes?limit=abc – 400 Bad Request",
      "request": {
        "method": "GET",
        "header": [],
        "url": "{{baseUrl}}/notes?limit=abc"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "type": "text/javascript",
            "exec": [
              "pm.test('Статус 400', function () { pm.response.to.have.status(400); });",
              "pm.test('Ошибка также возвращается в JSON', function () { pm.response.to.be.json; pm.expect(pm.response.json()).to.have.property('error'); });"
            ]
          }
        }
      ]
    },
    {
      "name": "GET /notes – 500 Internal Server Error (тестовый сервер)",
      "request": {
        "method": "GET",
        "header": [],
        "url": "{{errorBaseUrl}}/notes"
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "type": "text/javascript",
            "exec": [
              "pm.test('Статус 500', function () { pm.response.to.have.status(500); });",
              "pm.test('Сервер явно сообщает об ошибке в JSON', function () { pm.response.to.be.json; pm.expect(pm.response.json().error).to.be.a('string').and.not.empty; });"
            ]
          }
        }
      ]
    }
  ]
}
```

Приложение I. Окружение Postman

Файл: solutions/task5_api/Exam_API.postman_environment.json

```
{
  "id": "3b0f2999-69fc-4c32-a123-23e09af5f306",
  "name": "Экзамен – локально",
  "values": [
    { "key": "baseUrl", "value": "http://127.0.0.1:8000", "type": "default", "enabled": true },
    { "key": "errorBaseUrl", "value": "http://127.0.0.1:8001", "type": "default", "enabled": true }
  ],
  "_postman_variable_scope": "environment",
  "_postman_exported_at": "2026-10-07T12:00:00.000Z",
  "_postman_exported_using": "Postman/11.0"
}
```

Приложение J. Генератор документации

Файл: solutions/task6_docs/build_docx.py

```
#!/usr/bin/env python3
"""Заполняет приложенный шаблон документации API."""
from pathlib import Path
from docx import Document
from docx.shared import Inches, Pt, RGBColor
from docx.enum.table import WD_TABLE_ALIGNMENT, WD_CELL_VERTICAL_ALIGNMENT

ROOT = Path(__file__).resolve().parents[2]
TEMPLATE = ROOT / "Прил_6_03_КИМ_09.02.07-5-2027.docx"
OUTPUT = Path(__file__).resolve().parent / "API_документация.docx"

def set_cell(cell, text: str, bold: bool = False) -> None:
    cell.text = ""
    for index, line in enumerate(text.split("\n")):
        paragraph = cell.paragraphs[0] if index == 0 else cell.add_paragraph()
        paragraph.paragraph_format.space_after = Pt(2)
        run = paragraph.add_run(line)
        run.bold = bold
        run.font.size = Pt(8.5)
        cell.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER

def add_code(doc: Document, text: str) -> None:
    paragraph = doc.add_paragraph()
    paragraph.paragraph_format.left_indent = Inches(0.2)
    paragraph.paragraph_format.space_before = Pt(2)
    paragraph.paragraph_format.space_after = Pt(8)
    paragraph.paragraph_format.keep_together = True
    run = paragraph.add_run(text)
    run.font.name = "Consolas"
    run.font.size = Pt(8.5)
    run.font.color.rgb = RGBColor(31, 45, 61)

def main() -> None:
    doc = Document(TEMPLATE)
    doc.core_properties.title = "Документация API заметок"
    doc.core_properties.subject = "Задание 6 – описание разработанного API"
    doc.core_properties.author = "Участник демонстрационного экзамена"

    base_table = doc.tables[0]
    base_table.alignment = WD_TABLE_ALIGNMENT.CENTER
    set_cell(base_table.cell(0, 0), "Базовый URL", bold=True)
    base_table.cell(0, 1).merge(base_table.cell(0, 3))
    set_cell(base_table.cell(0, 1), "http://localhost:8000")
    set_cell(base_table.cell(1, 0), "Название метода", bold=True)
    set_cell(base_table.cell(1, 1), "Параметры", bold=True)
    set_cell(base_table.cell(1, 2), "Описание метода", bold=True)
    set_cell(base_table.cell(1, 3), "Формат ответа", bold=True)
    set_cell(base_table.cell(2, 0), "GET /notes\nGET /api/notes (алиас)")
    set_cell(base_table.cell(2, 1), "limit – необязательное целое число 1-1000; \nuser_id – необязательное положительное целое число.")
    set_cell(base_table.cell(2, 2), "Возвращает заметки из notes, присоединяя login автора из users. title_user = title + ' - ' + login; content без изменений; created at преобразуется в ДД.ММ.ГГГГ. Неизвестные, повторные и некорректные параметры дают 400. Если заметок нет, возвращается [].")
    set_cell(base_table.cell(2, 3), "200: массив JSON-объектов {id, title_user, content, formatted_date}. UTF-8; Content-Type: application/json.")
    for row in base_table.rows:
        for cell in row.cells:
            cell.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER

    status_table = doc.tables[1]
    set_cell(status_table.cell(0, 0), "Код", bold=True)
    set_cell(status_table.cell(0, 1), "Описание", bold=True)
    status_values = [
        ("200 OK", "Запрос обработан. Тело – JSON-массив; при отсутствии подходящих заметок – пустой массив []."),
        ("400 Bad Request", "Некорректный, повторный или неизвестный query-параметр; тело содержит поле error."),
        ("500 Internal Server Error", "Внутренняя ошибка/сбой БД; JSON вида {\"error\": \"Ошибка подключения к базе данных\"}.")
    ]
    for index, values in enumerate(status_values, start=1):
        set_cell(status_table.cell(index, 0), values[0])
        set_cell(status_table.cell(index, 1), values[1])
    extra = status_table.add_row().cells
    set_cell(extra[0], "405 Method Not Allowed")
    set_cell(extra[1], "Метод, отличный от GET, не поддерживается; заголовок Allow: GET, тело ошибки – JSON.")

    heading = doc.add_paragraph()
    heading.paragraph_format.space_before = Pt(10)
    run = heading.add_run("Примеры запросов и ответов")
    run.bold = True
    run.font.size = Pt(12)

    doc.add_paragraph("1. Успешное получение двух последних заметок:")
    add_code(doc, "GET http://localhost:8000/notes?limit=2\nAccept: application/json")
    add_code(doc, """HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

[
  {
    "id": 5,
    "title_user": "Итоги - admin",
    "content": "Сохранить результаты тестирования.",
    "formatted_date": "05.10.2026"
  },
  {
    "id": 4,
    "title_user": "Проверка базы - user",
    "content": "Убедиться, что внешние ключи включены.",
    "formatted_date": "04.10.2026"
  }
]""")

    doc.add_paragraph("2. Некорректный тип параметра:")
    add_code(doc, "GET http://localhost:8000/notes?limit=abc")
    add_code(doc, """HTTP/1.1 400 Bad Request
```

```

Content-Type: application/json; charset=utf-8

{"error": "Параметры limit и user_id должны быть положительными целыми числами; limit – от 1 до 1000."}""")
    doc.add_paragraph("3. Внутренняя ошибка базы данных (например, база недоступна):")
    add_code(doc, "GET http://localhost:8000/notes")
    add_code(doc, ""HTTP/1.1 500 Internal Server Error
Content-Type: application/json; charset=utf-8

{"error": "Ошибка подключения к базе данных"}""")
    doc.add_paragraph("Схема ответа одной заметки:")
    add_code(doc, ""{
    "id": 1,
    "title_user": "Конференция ИТ - user25",
    "content": "Текст заметки",
    "formatted_date": "15.03.2027"
}""")
    doc.add_paragraph("Параметры фильтрации не меняют формат объекта ответа. Порядок по умолчанию – от новых заметок к старым.")

OUTPUT.parent.mkdir(parents=True, exist_ok=True)
doc.save(OUTPUT)
print(OUTPUT)

if __name__ == "__main__":
    main()

```


Приложение К. Скрипт очистки и анализа

Файл: solutions/task7_analysis/clean_data.py

```
#!/usr/bin/env python3
"""Очистка исходного dataset_2026.xlsx и создание двух итоговых книг.

Требуется openpyxl (requirements.txt). Скрипт сохраняет все исходные строки,
исправляет типы/написания и строит лист с месячной выручкой и линейным графиком.
"""
from __future__ import annotations

import argparse
import hashlib
import re
from collections import defaultdict
from datetime import date, datetime
from decimal import Decimal, InvalidOperation
from pathlib import Path
from typing import Any

from openpyxl import Workbook, load_workbook
from openpyxl.chart import LineChart, Reference
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.worksheet.table import Table, TableStyleInfo

HERE = Path(__file__).resolve().parent
REPO_ROOT = HERE.parents[1]
HEADERS = ["transaction_id", "date", "product", "category", "quantity", "price", "shop", "manager"]
MONTHS_RU = [
    "Январь", "Февраль", "Март", "Апрель", "Май", "Июнь",
    "Июль", "Август", "Сентябрь", "Октябрь", "Ноябрь", "Декабрь",
]
PRODUCT_NAMES = {
    "утюг philips": "Утюг Philips",
    "микроволновая печь midea": "Микроволновая печь Midea",
    "чайник kitfort": "Чайник Kitfort",
    "пылесос dyson": "Пылесос Dyson",
    "пылесос dison": "Пылесос Dyson", # очевидная опечатка, есть совпадающие записи Dyson
    "стиральная машина bosch": "Стиральная машина Bosch",
    "телевизор samsung": "Телевизор Samsung",
    "телевизор samsung 4k": "Телевизор Samsung 4K",
    "холодильник lg": "Холодильник LG",
}
MANAGERS = {
    "иванов и.и.": "Иванов И.И.",
    "смирнова е.в.": "Смирнова Е.В.",
    "сидоров в.к.": "Сидоров В.К.",
    "петрова а.с.": "Петрова А.С.",
}

def parse_numeric(value: Any, field_name: str) -> Decimal:
    if value is None:
        raise ValueError(f"Пустое значение в поле {field_name}")
    if isinstance(value, (int, float, Decimal)):
        try:
            parsed = Decimal(str(value))
        except InvalidOperation as error:
            raise ValueError(f"Некорректное число {value!r} в поле {field_name}") from error
    else:
        text = str(value).replace("\u00a0", " ").strip()
        # Удаляем валюту/единицы измерения, оставляя цифры, знак и разделитель.
        text = re.sub(r"^[^0-9,.-]", "", text)
        if "." in text and "," in text:
            # В исходном наборе десятичная запятая не соседствует с разделителем тысяч;
            # при необходимости принимаем формат 1.234,56.
            text = text.replace(".", "").replace(",", ".")
        else:
            text = text.replace(",", ".")
        try:
            parsed = Decimal(text)
        except InvalidOperation as error:
            raise ValueError(f"Некорректное число {value!r} в поле {field_name}") from error
    if not parsed.is_finite():
        raise ValueError(f"Не конечное число в поле {field_name}: {value!r}")
    return parsed

def normalize_transaction_id(value: Any) -> str:
    text = str(value).strip().upper()
    match = re.fullmatch(r"TXN-?(\d+)", text)
    if not match:
        raise ValueError(f"Неизвестный формат transaction_id: {value!r}")
    return f"TXN-{int(match.group(1)):04d}"

def normalize_product(value: Any) -> str:
    text = " ".join(str(value or "").split())
    key = text.casefold()
    if key in PRODUCT_NAMES:
        return PRODUCT_NAMES[key]
    if not text:
        raise ValueError("Пустое название товара")
    return text

def infer_category(product: str) -> str:
    normalized = product.casefold()
    if normalized.startswith("пылесос"):
        return "Пылесосы"
    if normalized.startswith("телевизор"):
        return "Телевизоры"
    if normalized.startswith("стиральная машина"):
        return "Стиральные машины"
    if normalized.startswith("холодильник"):
        return "Холодильники"
    if normalized.startswith(("утюг", "чайник", "микроволновая печь")):
        return "Мелкая бытовая техника"
    raise ValueError(f"Для товара не настроена категория: {product}")
```

```

def normalize_manager(value: Any) -> str:
    text = " ".join(str(value or "").split())
    return MANAGERS.get(text.casefold(), text)

def as_excel_number(value: Decimal) -> int | float:
    return int(value) if value == value.to_integral_value() else float(value)

def load_and_clean(source: Path) -> tuple[list[dict[str, Any]], list[dict[str, Any]], int]:
    workbook = load_workbook(source, data_only=True, read_only=True)
    sheet = workbook.active
    all_rows = list(sheet.iter_rows(values_only=True))
    if not all_rows:
        raise ValueError("Исходная книга пуста")
    source_headers = list(all_rows[0][:8])
    if source_headers != HEADERS:
        raise ValueError(f"Ожидались колонки {HEADERS}, получены {source_headers}")

    raw_records: list[tuple[Any, ...]] = []
    for row in all_rows[1:]:
        # Лишние полностью пустые колонки (в исходной книге их три) игнорируются.
        first_eight = tuple(row[:8])
        if all(value is None for value in first_eight):
            continue
        if len(first_eight) != 8:
            raise ValueError(f"Строка не содержит 8 полей: {first_eight!r}")
        raw_records.append(first_eight)
    exact_duplicates = len(raw_records) - len(set(raw_records))

    cleaned: list[dict[str, Any]] = []
    change_log: list[dict[str, Any]] = []
    for excel_row, row in enumerate(raw_records, start=2):
        old = dict(zip(HEADERS, row))
        tx_id = normalize_transaction_id(old["transaction_id"])
        raw_date = old["date"]
        if isinstance(raw_date, datetime):
            trans_date = raw_date
        elif isinstance(raw_date, date):
            trans_date = datetime.combine(raw_date, datetime.min.time())
        else:
            trans_date = datetime.fromisoformat(str(raw_date).strip())
        if trans_date.year != 2026:
            raise ValueError(f"Ожидалась дата 2026 года, строка {excel_row}: {trans_date}")

        product = normalize_product(old["product"])
        category = infer_category(product)
        quantity_decimal = parse_numeric(old["quantity"], "quantity")
        if quantity_decimal <= 0 or quantity_decimal != quantity_decimal.to_integral_value():
            raise ValueError(f"Количество должно быть положительным целым числом, строка {excel_row}")
        price_decimal = parse_numeric(old["price"], "price")
        if price_decimal < 0:
            raise ValueError(f"Цена не может быть отрицательной, строка {excel_row}")
        shop = " ".join(str(old["shop"] or "").split())
        manager = normalize_manager(old["manager"])

        clean = {
            "transaction_id": tx_id,
            "date": trans_date,
            "product": product,
            "category": category,
            "quantity": int(quantity_decimal),
            "price": as_excel_number(price_decimal),
            "shop": shop,
            "manager": manager,
        }
        cleaned.append(clean)
        for field in HEADERS:
            previous = old[field]
            current = clean[field]
            if field == "date" and isinstance(previous, (datetime, date)):
                unchanged = previous.date() == current.date() if isinstance(previous, datetime) else previous ==
current.date()
            elif field in ("quantity", "price"):
                # Текст вида «3 шт.» / «294980 руб.» нужно отметить даже тогда,
                # когда извлеченное число совпало с итоговым числом.
                unchanged = isinstance(previous, (int, float, Decimal)) and not isinstance(previous, bool) \
                    and Decimal(str(previous)) == Decimal(str(current))
            else:
                unchanged = previous == current
            if not unchanged:
                change_log.append({
                    "excel_row": excel_row,
                    "transaction_id": tx_id,
                    "field": field,
                    "old_value": "" if previous is None else str(previous),
                    "new_value": "" if current is None else str(current),
                })

    ids = [row["transaction_id"] for row in cleaned]
    if len(ids) != len(set(ids)):
        raise ValueError("После стандартизации обнаружены повторяющиеся transaction_id")
    workbook.close()
    return cleaned, change_log, exact_duplicates

def style_table(sheet, header_row: int = 1) -> None:
    navy = "172B4D"
    for cell in sheet[header_row]:
        if cell.value is not None:
            cell.font = Font(bold=True, color="FFFFFF")
            cell.fill = PatternFill("solid", fgColor=navy)
            cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
    sheet.row_dimensions[header_row].height = 30
    sheet.freeze_panes = f"A{header_row + 1}"
    sheet.sheet_view.showGridLines = False

def add_excel_table(sheet, name: str, ref: str) -> None:
    table = Table(displayName=name, ref=ref)
    table.tableStyleInfo = TableStyleInfo(
        name="TableStyleMedium2", showFirstColumn=False,
        showLastColumn=False, showRowStripes=True, showColumnStripes=False,

```

```

)
sheet.add_table(table)

def write_clean_workbook(output: Path, cleaned: list[dict[str, Any]]) -> None:
    workbook = Workbook()
    sheet = workbook.active
    sheet.title = "Sheet1" # оставляем короткое стандартное имя листа для автопроверки
    sheet.append(HEADERS)
    for row in cleaned:
        sheet.append([row[field] for field in HEADERS])
    style_table(sheet)
    widths = [17, 14, 34, 30, 13, 14, 22, 22]
    for index, width in enumerate(widths, start=1):
        sheet.column_dimensions[chr(64 + index)].width = width
    for row in range(2, sheet.max_row + 1):
        sheet.cell(row, 2).number_format = "DD.MM.YYYY"
        sheet.cell(row, 5).number_format = "0"
        sheet.cell(row, 6).number_format = '#,##0.00;[Red]-#,##0.00'
    if cleaned:
        add_excel_table(sheet, "CleanTransactions", f"A1:H{sheet.max_row}")
    workbook.properties.title = "Очищенный набор транзакций розничной сети за 2026 год"
    workbook.properties.subject = "Задание 7 – предобработка без удаления строк"
    output.parent.mkdir(parents=True, exist_ok=True)
    workbook.save(output)

def write_analysis_workbook(
    output: Path,
    cleaned: list[dict[str, Any]],
    change_log: list[dict[str, Any]],
    exact_duplicates: int,
) -> dict[str, Any]:
    monthly = defaultdict(Decimal)
    data_revenues: list[Decimal] = []
    for row in cleaned:
        revenue = Decimal(str(row["quantity"])) * Decimal(str(row["price"]))
        data_revenues.append(revenue)
        monthly[row["date"].month] += revenue
    total = sum(monthly.values(), Decimal("0"))
    average = total / Decimal(12)
    min_month = min(range(1, 13), key=lambda month: monthly[month])
    max_month = max(range(1, 13), key=lambda month: monthly[month])

    workbook = Workbook()
    data_sheet = workbook.active
    data_sheet.title = "Данные"
    data_sheet.append(HEADERS + ["revenue"])
    for row, revenue in zip(cleaned, data_revenues):
        data_sheet.append([row[field] for field in HEADERS] + [as_excel_number(revenue)])
    style_table(data_sheet)
    for idx, width in enumerate([17, 14, 34, 30, 13, 14, 22, 22, 18], start=1):
        data_sheet.column_dimensions[chr(64 + idx)].width = width
    for row_num in range(2, data_sheet.max_row + 1):
        data_sheet.cell(row_num, 2).number_format = "DD.MM.YYYY"
        for col in (5, 6, 9):
            data_sheet.cell(row_num, col).number_format = '#,##0.00;[Red]-#,##0.00'
    add_excel_table(data_sheet, "AnalysisTransactions", f"A1:I{data_sheet.max_row}")

    month_sheet = workbook.create_sheet("Месячная выручка")
    month_headers = [
        "Месяц", "Выручка, руб.", "Доля годовой выручки, %",
        "Изменение к предыдущему месяцу, руб.", "Изменение к предыдущему месяцу, %",
        "Среднее за месяц, руб.",
    ]
    month_sheet.append(month_headers)
    for month in range(1, 13):
        value = monthly[month]
        previous = monthly[month - 1] if month > 1 else None
        change = value - previous if previous is not None else None
        change_pct = (change / previous * 100) if previous not in (None, Decimal("0")) else None
        month_sheet.append([
            MONTHS_RU[month - 1],
            as_excel_number(value),
            float(value / total * 100) if total else 0,
            as_excel_number(change) if change is not None else None,
            float(change_pct) if change_pct is not None else None,
            as_excel_number(average),
        ])
    style_table(month_sheet)
    for idx, width in enumerate([17, 20, 24, 36, 35, 24], start=1):
        month_sheet.column_dimensions[chr(64 + idx)].width = width
    for row_num in range(2, 14):
        for col in (2, 4, 6):
            month_sheet.cell(row_num, col).number_format = '#,##0.00;[Red]-#,##0.00'
        for col in (3, 5):
            month_sheet.cell(row_num, col).number_format = '0.00'
    add_excel_table(month_sheet, "MonthlyRevenue", "A1:F13")

    chart = LineChart()
    chart.title = "Динамика месячной выручки за 2026 год"
    chart.style = 13
    chart.y_axis.title = "Выручка, руб."
    chart.x_axis.title = "Месяц"
    chart.height = 10
    chart.width = 21
    chart.legend.position = "b"
    chart.y_axis.numFmt = '#,##0'
    revenue_data = Reference(month_sheet, min_col=2, max_col=2, min_row=1, max_row=13)
    average_data = Reference(month_sheet, min_col=6, max_col=6, min_row=1, max_row=13)
    categories = Reference(month_sheet, min_col=1, min_row=2, max_row=13)
    chart.add_data(revenue_data, titles_from_data=True)
    chart.add_data(average_data, titles_from_data=True)
    chart.set_categories(categories)
    chart.series[0].graphicalProperties.line.solidFill = "4058D6"
    chart.series[0].graphicalProperties.line.width = 28575
    chart.series[1].graphicalProperties.line.solidFill = "E0565B"
    chart.series[1].graphicalProperties.line.width = 19050
    chart.series[1].graphicalProperties.line.prstDash = "dash"
    month_sheet.add_chart(chart, "H2")
    month_sheet.freeze_panes = "A2"

```

```

summary = workbook.create_sheet("Итоги")
summary.append(["Показатель", "Значение"])
metrics = [
    ("Строк в исходном наборе", len(cleaned)),
    ("Полных дубликатов", exact_duplicates),
    ("Строк удалено", 0),
    ("Годовая выручка, руб.", as_excel_number(total)),
    ("Средняя месячная выручка, руб.", as_excel_number(average)),
    ("Месяц максимальной выручки", MONTHS_RU[max_month - 1]),
    ("Максимальная месячная выручка, руб.", as_excel_number(monthly[max_month])),
    ("Месяц минимальной выручки", MONTHS_RU[min_month - 1]),
    ("Минимальная месячная выручка, руб.", as_excel_number(monthly[min_month])),
    ("Размах месячной выручки, руб.", as_excel_number(monthly[max_month] - monthly[min_month])),
    ("Строк с зафиксированными исправлениями", len({x["excel_row"] for x in change_log})),
    ("Количество исправлений по полям", len(change_log)),
]
for item in metrics:
    summary.append(list(item))
summary.append(["Интерпретация", "Выручка волатильна: максимум – ноябрь, минимум – сентябрь; устойчивого
ежемесячного роста нет."])
summary.append(["Проверить по первичному источнику", "TXN-0001: 897270 руб./шт. за утюг Philips – явный ценовой
выброс. В clean-файле сохранён как получен: в приложенных условиях нет подтверждённого исправленного значения."])
style_table(summary)
summary.column_dimensions["A"].width = 43
summary.column_dimensions["B"].width = 92
summary.freeze_panes = "A2"

log_sheet = workbook.create_sheet("Журнал очистки")
log_headers = ["Строка Excel", "transaction_id", "Поле", "Исходное значение", "Очищенное значение"]
log_sheet.append(log_headers)
for item in change_log:
    log_sheet.append([item["excel_row"], item["transaction_id"], item["field"], item["old_value"], item[
"new_value"]])
style_table(log_sheet)
for col, width in {"A": 15, "B": 18, "C": 18, "D": 38, "E": 38}.items():
    log_sheet.column_dimensions[col].width = width
if change_log:
    add_excel_table(log_sheet, "CleaningLog", f"A1:E{log_sheet.max_row}")

workbook.properties.title = "Анализ продаж за 2026 год"
workbook.properties.subject = "Месячная выручка, среднее значение и временной ряд"
output.parent.mkdir(parents=True, exist_ok=True)
workbook.save(output)
return {
    "monthly": monthly,
    "total": total,
    "average": average,
    "min_month": min_month,
    "max_month": max_month,
    "change_log": change_log,
    "duplicates": exact_duplicates,
}

def main() -> None:
    parser = argparse.ArgumentParser(description=_doc_)
    parser.add_argument("--input", type=Path, default=REPO_ROOT / "dataset_2026.xlsx")
    parser.add_argument("--clean-output", type=Path, default=HERE / "clean_dataset_2026.xlsx")
    parser.add_argument("--analysis-output", type=Path, default=HERE / "analysis_2026.xlsx")
    args = parser.parse_args()

    cleaned, change_log, duplicate_count = load_and_clean(args.input)
    write_clean_workbook(args.clean_output, cleaned)
    result = write_analysis_workbook(args.analysis_output, cleaned, change_log, duplicate_count)
    digest = hashlib.sha256(args.clean_output.read_bytes()).hexdigest()
    print(f"Строк в очищенном файле: {len(cleaned)}; удалено: 0; полных дубликатов: {duplicate_count}")
    print(f"Исправлений в журнале: {len(change_log)}; затронуты строки: {len({item['excel_row'] for item in
change_log})}")
    print(f"Годовая выручка: {result['total']}; средняя за месяц: {result['average']}")
    print(f"Максимум: {MONTHS_RU[result['max_month'] - 1]} – {result['monthly'][result['max_month']]}")
    print(f"Минимум: {MONTHS_RU[result['min_month'] - 1]} – {result['monthly'][result['min_month']]}")
    print(f"SHA-256 файла очистки (фингерпринт, не контрольный код приложения): {digest}")
    print(f"Очищенный набор: {args.clean_output}")
    print(f"Расчёты и график: {args.analysis_output}")

if __name__ == "__main__":
    main()

```

Приложение L. Локальная предварительная проверка задания 7

Файл: solutions/task7_analysis/preflight_check.py

```
#!/usr/bin/env python3
"""Локальная предварительная проверка файлов задания 7.
```

Это НЕ официальный Analytics_autotest и не выдаёт его контрольный код. Скрипт проверяет требования, которые явно перечислены в приложенном ТЗ: сохранность 30 строк, типы/нормализацию, расчёт выручки, месячную сводку, итоги и наличие графика с серией среднего значения.

Запуск из корня репозитория:
python3 solutions/task7_analysis/preflight_check.py

В комплекте заданий можно запускать из этой папки без параметров, если рядом находится исходный dataset_2026.xlsx.

```
from __future__ import annotations

import argparse
import sys
from datetime import date, datetime
from decimal import Decimal, InvalidOperation
from pathlib import Path
from typing import Any

from openpyxl import load_workbook
from openpyxl.chart import LineChart

from clean_data import infer_category, normalize_manager, normalize_product, normalize_transaction_id

HERE = Path(__file__).resolve().parent
REPO_ROOT = HERE.parents[1]
DEFAULT_SOURCE = HERE / "dataset_2026.xlsx"
if not DEFAULT_SOURCE.is_file():
    DEFAULT_SOURCE = REPO_ROOT / "dataset_2026.xlsx"
DEFAULT_CLEAN = HERE / "clean_dataset_2026.xlsx"
DEFAULT_ANALYSIS = HERE / "analysis_2026.xlsx"
HEADERS = ["transaction_id", "date", "product", "category", "quantity", "price", "shop", "manager"]
MONTHS = [
    "Январь", "Февраль", "Март", "Апрель", "Май", "Июнь",
    "Июль", "Август", "Сентябрь", "Октябрь", "Ноябрь", "Декабрь",
]

EXPECTED = {
    "Строк в исходном наборе": 30,
    "Полных дубликатов": 0,
    "Строк удалено": 0,
    "Годовая выручка, руб.": 7_759_680,
    "Средняя месячная выручка, руб.": 646_640,
    "Месяц максимальной выручки": "Ноябрь",
    "Максимальная месячная выручка, руб.": 1_794_540,
    "Месяц минимальной выручки": "Сентябрь",
    "Минимальная месячная выручка, руб.": 127_949,
    "Размах месячной выручки, руб.": 1_666_591,
    "Строк с зафиксированными исправлениями": 27,
    "Количество исправлений по полям": 47,
}

def dec(value: Any) -> Decimal | None:
    if value is None or isinstance(value, bool):
        return None
    try:
        result = Decimal(str(value))
    except (InvalidOperation, ValueError, TypeError):
        return None
    return result if result.is_finite() else None

def check(condition: bool, label: str, failures: list[str]) -> None:
    print(f"[{'OK' if condition else 'ОШИБКА'}] {label}")
    if not condition:
        failures.append(label)

def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--source", type=Path, default=DEFAULT_SOURCE)
    parser.add_argument("--clean", type=Path, default=DEFAULT_CLEAN)
    parser.add_argument("--analysis", type=Path, default=DEFAULT_ANALYSIS)
    args = parser.parse_args()
    failures: list[str] = []

    for label, path in (("Исходный набор", args.source), ("Очищенный набор", args.clean), ("Книга анализа", args.analysis)):
        check(path.is_file(), f"{label}: файл существует ({path})", failures)
        if failures:
            print("Проверка остановлена: не найдены входные/выходные файлы.")
            return 1

    source_wb = load_workbook(args.source, data_only=True, read_only=True)
    clean_wb = load_workbook(args.clean, data_only=True, read_only=True)
    analysis_wb = load_workbook(args.analysis, data_only=True, read_only=False)
    try:
        source_rows_all = list(source_wb.active.iter_rows(values_only=True))
        source_headers = list(source_rows_all[0][:8]) if source_rows_all else []
        source_rows = [tuple(row[:8]) for row in source_rows_all[1:]] if any(v is not None for v in source_headers) else []
        clean_sheet = clean_wb.active
        clean_rows_all = list(clean_sheet.iter_rows(values_only=True))
        clean_headers = list(clean_rows_all[0][:8]) if clean_rows_all else []
        clean_rows = [tuple(row[:8]) for row in clean_rows_all[1:]] if any(v is not None for v in clean_headers) else []

        check(source_headers == HEADERS, "Исходные заголовки совпадают с заданием", failures)
        check(clean_headers == HEADERS, "Очищенная книга сохраняет исходные 8 колонок", failures)
        check(len(source_rows) == EXPECTED["Строк в исходном наборе"], "В источнике ровно 30 транзакций", failures)
        check(len(clean_rows) == len(source_rows), "Число строк сохранено: ни одна транзакция не потеряна", failures)
        duplicate_count = len(clean_rows) - len(set(clean_rows))
    except Exception as e:
        print(f"Ошибка при проверке: {e}")
        return 1

    return 0
```

```

check(duplicate_count == 0, "Полных дубликатов после очистки нет", failures)

ids: list[str] = []
dates_ok = True
numbers_ok = True
canonical_ok = len(source_rows) == len(clean_rows)
for row, row in zip(source_rows, clean_rows):
    tx_id, tx_date, product, category, quantity, price, shop, manager = row
    ids.append(str(tx_id))
    dates_ok &= isinstance(tx_date, (datetime, date)) and tx_date.year == 2026
    q, p = dec(quantity), dec(price)
    numbers_ok &= q is not None and p is not None and q > 0 and q == q.to_integral_value() and p >= 0
    expected_product = normalize_product(row[2])
    expected_manager = normalize_manager(row[7])
    canonical_ok &= str(tx_id) == normalize_transaction_id(row[0])
    canonical_ok &= str(product) == expected_product
    canonical_ok &= str(category) == infer_category(expected_product)
    canonical_ok &= str(shop) == " ".join(str(row[6] or "").split())
    canonical_ok &= str(manager) == expected_manager
check(len(ids) == len(set(ids)) and all(len(tx_id) == 8 for tx_id in ids), "ID транзакций нормализованы и уникальны", failures)
check(dates_ok, "Все даты – настоящие даты 2026 года", failures)
check(numbers_ok, "Количество и цена – числовые; количество целое и положительное", failures)
check(canonical_ok, "Названия, категории, магазины и менеджеры приведены к единому виду", failures)

monthly = [Decimal("0") for _ in range(12)]
line_revenues: list[Decimal] = []
for row in clean_rows:
    tx_date, quantity, price = row[1], dec(row[4]), dec(row[5])
    revenue = quantity * price # type: ignore[operator]
    line_revenues.append(revenue)
    monthly[tx_date.month - 1] += revenue
total = sum(monthly, Decimal("0"))
average = total / Decimal(12)
max_index = max(range(12), key=lambda i: monthly[i])
min_index = min(range(12), key=lambda i: monthly[i])
check(total == Decimal("7759680"), f"Годовая выручка рассчитана верно: {total}", failures)

required_sheets = {"Данные", "Месячная выручка", "Итоги", "Журнал очистки"}
check(required_sheets.issubset(set(analysis_wb.sheetnames)), "В книге анализа есть листы данных, месяцев, итогов и журнала", failures)
if required_sheets.issubset(set(analysis_wb.sheetnames)):
    data_sheet = analysis_wb["Данные"]
    data_values = list(data_sheet.iter_rows(min_row=2, values_only=True))
    data_ok = len(data_values) == len(clean_rows)
    for row, clean, expected_revenue in zip(data_values, clean_rows, line_revenues):
        data_ok &= str(row[0]) == str(clean[0])
        actual = dec(row[8]) if len(row) > 8 else None
        data_ok &= actual == expected_revenue
    check(data_ok, "Выручка каждой строки равна quantity × price", failures)

    month_sheet = analysis_wb["Месячная выручка"]
    month_names = [month_sheet.cell(row, 1).value for row in range(2, 14)]
    actual_months = [dec(month_sheet.cell(row, 2).value) for row in range(2, 14)]
    actual_shares = [dec(month_sheet.cell(row, 3).value) for row in range(2, 14)]
    actual_average = [dec(month_sheet.cell(row, 6).value) for row in range(2, 14)]
    months_ok = month_names == MONTHS and actual_months == monthly
    expected_shares = [value / total * Decimal("100") for value in monthly]
    shares_ok = (
        all(value is not None for value in actual_shares)
        and all(abs(actual - expected) <= Decimal("0.01") for actual, expected in zip(actual_shares,
expected_shares))
        and abs(sum(actual_shares, Decimal("0")) - Decimal("100")) <= Decimal("0.02")
    )
    avg_ok = all(value is not None and abs(value - average) < Decimal("0.01") for value in actual_average)
    check(months_ok, "Помесячная выручка совпадает с датами и суммами транзакций", failures)
    check(shares_ok, "Доля каждого месяца рассчитана от годовой выручки; сумма долей – 100%", failures)
    check(avg_ok, "Среднее значение указано для всех 12 месяцев", failures)
    charts = month_sheet.charts
    chart_ok = False
    if len(charts) == 1 and isinstance(charts[0], LineChart) and len(charts[0].series) == 2:
        chart = charts[0]
        categories = [series.cat.numRef.f if series.cat and series.cat.numRef else None for series in chart.series]
        revenues = [series.val.numRef.f if series.val and series.val.numRef else None for series in chart.series]
        expected_categories = "'Месячная выручка'!$A$2:$A$13"
        chart_ok = categories == [expected_categories, expected_categories] and revenues == [
            "'Месячная выручка'!$B$2:$B$13",
            "'Месячная выручка'!$F$2:$F$13",
        ]
    check(chart_ok, "Линейный график использует месяцы по порядку, выручку и серию среднего", failures)

    summary = analysis_wb["Итоги"]
    metrics = {summary.cell(row, 1).value: summary.cell(row, 2).value for row in range(2, summary.max_row + 1)}
    metrics_ok = True
    for key, expected in EXPECTED.items():
        actual = metrics.get(key)
        if isinstance(expected, int):
            metrics_ok &= dec(actual) == Decimal(expected)
        else:
            metrics_ok &= actual == expected
    metrics_ok &= month_names[max_index] == "Ноябрь" and monthly[max_index] == Decimal("1794540")
    metrics_ok &= month_names[min_index] == "Сентябрь" and monthly[min_index] == Decimal("127949")
    check(metrics_ok, "Итоговые контрольные показатели и журнал совпадают с набором задания", failures)
    log_sheet = analysis_wb["Журнал очистки"]
    check(log_sheet.max_row == 48, "Журнал содержит заголовки и все 47 исправлений", failures)

# Не корректируем данные без подтверждения первичного источника; лишь убеждаемся,
# что зафиксированный выброс не потерян при экспорте.
outlier = next((row for row in clean_rows if str(row[0]) == "TXN-0001"), None)
check(outlier is not None and dec(outlier[5]) == Decimal("897270"), "TXN-0001 сохранён как отмеченный выброс без выдуманной подстановки", failures)
finally:
    source_wb.close()
    clean_wb.close()
    analysis_wb.close()

print("\nЛокальная предварительная проверка завершена.")
print("Это не официальный Analytics_autotest и не выдаёт контрольный код 7.1.")
if failures:

```

```
        print(f"Провалено проверок: {len(failures)}")
        return 1
    print("Все проверки, которые можно вывести из приложенного ТЗ, пройдены.")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```